课题编号：2023YFC3305503

银行信息系统抗量子/混合公钥证书生成工具

所属课题：银行业典型信息系统抗量子密码安全协议迁移技术

所属项目：银行业及其关键基础设施信息系统的抗量子密码迁移技术研究

所属专项：社会治理与智慧社会科技支撑（平安中国）

编制单位：中国科学院信息工程研究所

2026 年 7 月 2 日

抗量子证书生成工具
设计报告

编　　写 _____

校　　对 _____

批　　准 _____

文档控制

变更记录

版本号	创建/修改日期	版本控制人	增加/修改/删除	简单描述
V1.0	2025.4.6	刘凯博	增加	设计报告框架及调研工作撰写
V1.1	2026.6..6	刘凯博	增加	调研方案完善
V1.2	2026.7.6	刘凯博	增加/修改	设计方案完善
V1.1	2026.7.9	张航	修改	文档格式修改

目录

1. 设计目标 1
2. 背景现状 2
2.1. 概述 2
2.2. 抗量子签名算法研究现状 2
2.2.1. 基于格的签名算法研究现状 2
2.2.2. 基于哈希的签名算法研究现状 3
2.2.3. 基于多变量多项式的签名算法研究现状 4
2.2.4. 基于超椭圆曲线的签名算法 4
2.3. 混合数字证书研究现状 5
2.3.1. TLS1.3 中混合认证模式研究 5
2.3.2. 混合证书链设计及实现 8
2.3.3. 多种证书混合模式设计研究 10
2.3.4. Google 混合认证方式 11
2.4. 国内外抗量子证书研究现状 12
3. 数字证书研究基础 16
3.1. 国密算法与证书格式 17
3.1.1. 版本 (Version) 18
3.1.2. 序列号 (Serial Number) 18
3.1.3. 签名算法标识符 (Signature Algorithm Identifier) 18
3.1.4. 签发人名称 (Issuer Name) 19
3.1.5. 有效期 (Validity Period) 19
3.1.6. 主体名称 (Subject Name) 19
3.1.7. 主体公钥信息 (Subject Public Key Info) 19
3.1.8. 扩展字段 (Extensions) 19
3.2. 数字证书的生命周期 20
3.2.1. 证书的生命 21
3.2.2. 证书的使用 22
3.2.3. X.509 证书的算法 28
4. 方案设计 30
4.1. 混合签名设计 31
4.2. 混合证书格式设计 33
4.3. 公钥证书链设计 34
4.4. 抗量子证书生成工具设计 35
5. 抗量子证书生成工具模块设计 37
5.1. 系统整体分层架构 37
5.2. 各层级模块详细功能设计 38
5.2.1. 公共工具支撑层 38
5.2.2. 密码算法支撑层 39
5.2.3. 证书生成层 39
5.2.4. 接口调度层 40

5.3. 模块整体执行流程 41
6. 抗量子证书生成工具实现 41
6.1. 整体架构 42
6.2. 数据结构和基础算法 42
6.3. 抗量子密码生成模块 43
6.4. 分层证书签发实现 46
6.5. 主程序调度与证书链验证 49
6.6. 小结 50
7. 参考文献 51

图目录

图 1 抗量子 TLS1.3 握手流程.....6
图 2 抗量子迁移 X.509 证书格式.....7
图 3 握手时延对比.....8
图 4 迁移后的结果.....10
图 5（非）分离混合签名方案.....10
图 6 混合签名：经典和 PQC 方案的强嵌套.....12
图 7 混合连接模式证书结构.....14
图 8 混合绑定模式证书结构.....14
图 9 混合使用拓展证书格式.....15
图 10 混合差分拓展证书结构.....15
图 11 X.509 证书格式.....17
图 12 证书全生命周期.....21
图 13 Bob 证书的验证.....23
图 14 归档证书的用途.....28
图 15 可分离混合签名.....32
图 16 不可分离混合签名.....33
图 17 混合证书结构.....34
图 18 混合证书链结构设计.....34
图 19 抗量子/混合公钥证书生成工具架构图.....37
图 20 抗量子证书生成工具整体架构图.....38

表目录

表 1 不同签名算法的证书尺寸.....7
表 2 混合迁移算法路线.....9

1. 设计目标

数字证书作为各类安全协议实现身份认证的主流载体，已在 TLS、IPSec、SSH 等协议中大规模使用，目前主流数字证书无论遵循国际 X.509 标准还是我国 GB/T 20518-2018 国密双证书规范，均依托 RSA、ECDSA、SM2 等传统公钥算法完成签名核验与身份鉴别，而上述算法的安全基础建立在大整数分解、椭圆曲线离散对数难题之上，具备强大算力的量子计算机可在短时间内攻破该类密码体系，一旦量子计算规模化落地，攻击者既能伪造合法数字证书、在安全协议握手环节发起中间人攻击，窃取通信交互中的身份凭证与传输数据，也能解密长期留存的加密会话记录，彻底瓦解现有基于数字证书的身份认证信任体系。由此可见，数字证书体系的抗量子改造是全系列安全协议完成抗量子安全迁移过程中不可或缺的核心环节。

基于此，本课题针对性研制开发一套抗量子数字证书生成工具。在工具设计与研发过程中，兼需要顾过渡场景适配、标准规范兼容与实际落地易用性多重核心需求。一方面，工具需同时支持经典公钥算法与抗量子密码算法，支持传统证书与抗量子证书/混合证书的一体化生成，满足安全协议平滑抗量子迁移的过渡应用场景；另一方面，针对抗量子证书的混合签名机制、多算法嵌套方式、自定义扩展字段适配等关键技术规范尚未形成统一标准。为此，本课题研发的工具严格遵循 X.509 国际证书标准与 GB/T 20518-2018 国密证书规范设计开发，生成的抗量子证书完全贴合现有证书数据结构与编码规则，可无缝适配传统 PKI 架构以及 TLS、IPSec、SSH 等主流安全协议的解析与校验机制，无需对现有网络安全体系进行大规模改造，大幅降低抗量子证书的落地部署与运维成本。在功能覆盖层面，本工具最终实现两类主流抗量子密码算法的集成适配，全面覆盖 128 比特、256 比特两种主流量子安全等级，可适配不同安全等级的网络业务场景，为各类安全协议的抗量子迁移提供便捷、规范、可靠的证书生成支撑。

2. 背景现状

2.1. 概述

随着量子计算技术的快速发展，传统的公钥算法如 RSA 和 ECC 面临着严重威胁。量子计算机利用 Shor 算法[1]可在多项式时间内破解这些算法，使得加密数据面临被实时解密和未来的“先收集后解密”攻击的威胁。因此，为了适应量子计算时代的挑战，混合证书应运而生，结合了 X.509 证书[2]和国密证书，同时支持传统签名算法和抗量子签名算法，确保未来的通信安全。

抗量子证书生成工具通过结合 RSA 签名[3]、SM2 签名和 ML-DSA 签名[34]，为数据安全提供了双重保障。传统的 RSA 和 SM2 签名算法依然能在当前环境中有效工作，而 ML-DSA 则为未来的量子计算机攻击提供防护。此外，国密标准下的 SM2 签名和公钥加密算法，结合 ML-DSA 的抗量子特性，确保了混合证书能够在全球范围内为不同的网络安全需求提供支持，尤其是针对中国网络安全的需求。

该混合证书生成工具不仅支持传统公钥算法，还通过扩展字段为加密证书和签名证书提供了抗量子加密的能力。在加密证书中，Key Usage 扩展字段将包括 Key Encipherment 和 Data Encipherment，确保数据交换的加密安全。而签名证书中的 Key Usage 将包括 Digital Signature 和 Non Repudiation，确保签名和不可否认性。同时，在 X.509v3 扩展字段中加入了 OQS_PUBLIC_KEY 和 OQS_SIGNATURE 的自定义 OID，用于标识和存储抗量子公钥和签名信息。

通过这种混合证书格式的设计和抗量子证书生成工具的设计实现，企业能够在保持现有基础设施兼容性的同时，逐步过渡到量子安全的体系结构，确保数据在未来量子计算环境中的长期安全性。这不仅是量子时代的必要准备，也是数字基础设施长期稳定运行的必备条件。

2.2. 抗量子签名算法研究现状

2.2.1. 基于格的签名算法研究现状

基于格的签名算法的研究始于 1996 年，Goldreich、Goldwasser 和 Halevi 首次提出了基于格的密码体制[4]，这为日后抗量子密码学的发展奠定了理论基础。

格的复杂结构和数学性质为抗量子攻击提供了天然的优势。2008 年，Gentry、Peikert 和 Vaikuntanathan 提出了第一个具有抗量子攻击能力的基于格的公钥加密和签名方案[5]，进一步证实了基于格的方案在抗量子密码学中的潜力。这一研究推动了更多基于格的签名方案的诞生。2011 年，Lyubashevsky 提出了一种新型的基于格的签名方案，即 Lyubashevsky 签名方案[6]。该方案利用了格中的随机化技术，通过“拒绝采样”减少了签名暴露信息的风险，使得签名的计算过程更加高效和安全，这一方案在当时成为格签名研究中的重要进展。2012 年，Ducas 等人[7]提出了 BLISS（Bimodal Lattice Signature Scheme）签名方案，成为格签名算法的又一个里程碑。BLISS 采用了双模高斯采样的技巧，既降低了签名方案的密钥和签名大小，又在抗量子安全性和计算效率之间取得了更好的平衡。然而，由于算法复杂度较高，BLISS 在实际应用中仍存在一定的局限性，但其设计思想影响了后续算法的研究。

近年来，随着抗量子密码学需求的提升，基于格的签名算法受到了更多关注。2017 年，美国国家标准与技术研究院（NIST）启动了后量子密码算法的标准化项目，吸引了全球的研究力量。众多基于格的签名算法进入了 NIST 标准化的候选名单，其中最为著名的包括 Dilithium 和 FALCON。Dilithium 基于模块格结构，具有良好的计算效率和相对较小的签名与密钥大小，其结构相对简单，便于实现。FALCON 则采用了最优最小模误差（NTRU 格）的数学结构，具有出色的性能表现，特别是在签名的空间效率上表现突出。这两种算法都进入了 NIST 标准化的最终阶段，并被认为是适合未来抗量子应用的有力候选。

2.2.2. 基于哈希的签名算法研究现状

Ralph Merkle 在 1979 年提出的 Merkle 签名方案拉开了序幕，这是第一个基于哈希函数的公钥签名算法。Merkle 签名方案具有良好的安全性，基于哈希函数的抗量子计算能力使其不易受到量子计算攻击[8]。然而，它的主要缺点在于需要较大的签名和公钥尺寸，使其在实际应用中受到一定限制。随着研究的深入，改进 Merkle 方案的尝试逐渐增多。20 世纪 90 年代，Lamport 签名方案[9]被提出，虽然其签名效率高，但每个密钥只能使用一次，这导致其在多次使用场景下不适用。随后，Winternitz 哈希链在 Merkle 签名的基础上进行了改进，通过优化签名生成过程，提升了整体的计算效率和存储效率[10]。进入 21 世纪，LMS（Leighton-

Micali Signatures）和 XMSS（eXtended Merkle Signature Scheme）等新算法应运而生。LMS 作为一种轻量级方案，保持了较低的复杂度和较高的安全性，被纳入 NIST 标准化过程[11]。XMSS 进一步优化了 Merkle 签名的使用体验，通过引入固定密钥大小和增强的抗量子攻击能力，解决了传统方案的一次性密钥问题，并得到了广泛认可[12]。

近年来，SPHINCS+等无状态哈希签名方案的出现进一步推动了研究的发展。SPHINCS+结合了多层 Merkle 树和伪随机函数，克服了早期方案的状态管理问题，并在安全性和效率之间取得了平衡。其作为一种抗量子计算的签名算法，成为未来公钥加密系统的重要候选之一。

2.2.3. 基于多变量多项式的签名算法研究现状

基于多变量多项式的签名算法是一类抗量子计算的公钥密码方案，其历史可追溯到1988年，由Matsumoto和Imai提出的C*方案拉开了这一领域的序幕[13]。尽管 C*方案在理论上具有较高的效率，但 1995 年被 Patarin 攻破，暴露出结构性弱点[14]。在 1996 年，Patarin 提出了 HFE（Hidden Field Equations）方案[15]，旨在通过隐藏域的复杂变换提高安全性。HFE 引入了更复杂的多项式结构来抵抗攻击，但仍遭遇了几种特定的攻击，如 2002 年 Faugère 和 Joux 提出的 MinRank 攻击[16]。进入 2000 年代，多变量公钥密码系统不断发展。2002 年，Tsuyoshi Takagi 和 Hideki Miyahara 等人提出了 UOV（Unbalanced Oil and Vinegar）方案[17]，它通过将变量分成“油”和“醋”两部分来提高攻击难度，避免了某些结构性弱点。UOV 算法的变种 Rainbow 方案[18]在 2005 年由 Ding 等人提出，其多层结构进一步增强了抗攻击能力，并成为该领域的重要方案。随着量子计算的发展，NIST 启动了抗量子密码标准化竞赛。Rainbow 成为了候选方案之一，并受到了极大关注。然而，2020 年，一些学者提出了针对 Rainbow 的新型攻击方法，使其安全性受到质疑。

近年来，研究者们不断探索新的多变量方案，如 Gui 和 QUARTZ，尝试优化签名效率和安全性之间的平衡。2022 年，围绕改进 UOV 和 HFE 算法的研究依然活跃，目标是构建在计算复杂度上具有更高抗量子攻击能力的签名算法。

2.2.4. 基于超椭圆曲线的签名算法

基于超椭圆曲线的签名算法是密码学领域的重要研究方向之一，旨在提供安全高效的数字签名方案。20 世纪 90 年代初期，椭圆曲线密码学已经受到广泛关注，人们逐渐将视野拓展到更一般的超椭圆曲线以实现更高的安全性和更小的密钥尺寸。超椭圆曲线是椭圆曲线的推广，定义上更为复杂，因此理论上提供了更高的安全性。1994 年，Semaev、Koblitz 等人开始研究超椭圆曲线在密码学中的应用，提出了相关的数学基础和计算技术，为后续基于超椭圆曲线的签名算法奠定了理论基础。2000 年初，具体的基于超椭圆曲线的签名算法开始被提出，例如 Schnorr 型[19]和 DSS 型[20]签名的推广，这些方案利用超椭圆曲线提供与椭圆曲线方案类似的计算效率，但提高了抗攻击能力。随着量子计算对传统公钥密码系统的威胁，研究者们在 21 世纪加大了对基于超椭圆曲线的抗量子算法的探索。由于超椭圆曲线上的离散对数问题在某些情况下比椭圆曲线更难解决，它们被视为未来抗量子密码学研究的潜在方向之一。

近年来，研究人员提出了新的签名方案，结合超椭圆曲线和其他数学结构以增强安全性和效率。例如，通过结合多变量和格基技术，超椭圆曲线签名算法实现了更复杂的结构，以应对未来量子计算的威胁。研究还在持续，以提高方案的可扩展性和实际应用中的性能。

2.3. 混合数字证书研究现状

X.509 证书在当前数字认证体系中占据核心地位，其灵活的架构为安全技术的不断演进提供了良好的支持。通过引入多算法的签名设计，抗量子迁移的 X.509 证书不仅能兼容传统的经典加密算法（如 RSA、ECC），也能集成抗量子签名算法（如 Dilithium、SPHINCS+），实现同时支持经典和抗量子验证的功能。在设计中，既可以采用可分离模式实现经典签名和抗量子签名的独立性，又可以采用不可分离模式通过嵌套设计增强安全性。这种灵活性使得证书可以在传统与量子安全方案间平稳过渡，同时为实际部署提供更多选择。抗量子 X.509 证书的高兼容性和扩展性，不仅降低了整体迁移成本，还增强了抵御未来潜在攻击的能力，为后量子时代的安全认证体系奠定了坚实基础。以下是对于目前针对 X.509 证书的迁移情况和 TLS 协议上的应用情况。

2.3.1. TLS1.3 中混合认证模式研究

[21]对 NIST 候选签名算法进行了详细的性能评估，并在真实网络条件下分析了这些算法对 TLS 1.3 连接建立延迟的影响。此外，我们还探讨了后量子算法对 TLS 会话吞吐量的影响，并分析了长后量子签名与计算负担较重的后量子密码操作之间的权衡关系。研究结果表明，至少两种后量子签名算法的采用在额外开销较少的情况下是可行的，其性能与当前的签名算法相差不大。许多 NIST 后量子候选算法可以有效地应用于对时间敏感性要求较低的场景。最后，提出并评估了在 TLS 证书链中结合使用不同后量子签名算法的方案。结果显示，与使用单一后量子签名算法相比，这种方案显著减少了 TLS 握手时间，并大幅提高了服务器 TLS 隧道连接的处理速率。

如图 1 所示使用抗量子（Post-Quantum, PQ）X.509 证书的 TLS 握手协议在流程上与传统 TLS 1.3 类似，但关键环节引入了抗量子算法以增强安全性。首先，客户端通过 ClientHello 消息表明支持的抗量子加密套件和签名算法，服务器响应 ServerHello 消息，选择合适的抗量子密钥交换算法（如 Kyber）并提供采用后量子签名算法（如 Dilithium、Falcon）签名的 X.509 证书链。随后，客户端验证服务器证书链的完整性和签名的正确性，并通过后量子密钥交换算法协商生成共享主密钥（Pre-Master Secret）。完成握手后，双方使用协商的共享密钥加密后续通信。

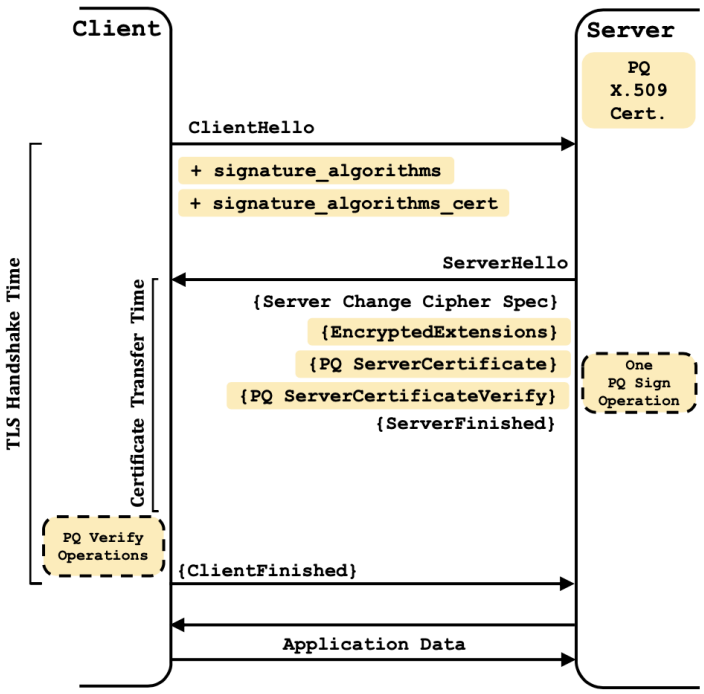


图 1 抗量子 TLS1.3 握手流程

如图 2 所示，X.509 证书的抗量子迁移需要对签名算法、公钥和签名域进行修改。在签名算法方面，将传统算法（如 RSA 或 ECDSA）替换为后量子签名算法（如 Dilithium、Falcon 或 SPHINCS+）；签名域的 signatureValue 字段将存储后量子签名值，这些签名通常比传统签名更大，可能增加证书的整体大小。

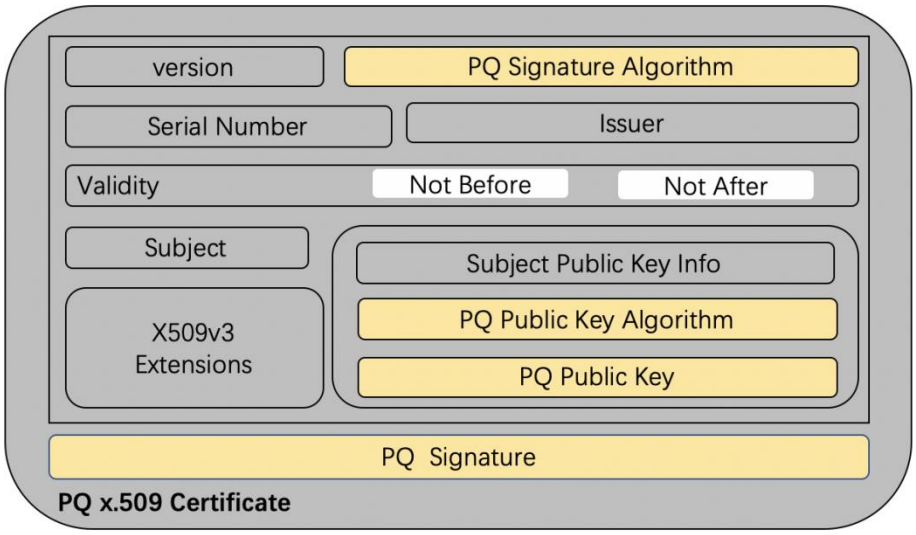


图 2 抗量子迁移 X. 509 证书格式

如表 1 及图 3 所示，这项研究使用了来自 liboqs 的测试脚本[22][23]。表 1 展示了在本研究使用的机器上运行的各种方案的平均签名和验证时间。由于这些操作的持续时间会直接影响实验中的 TLS 握手时间，因此我们测量了这些算法在各类机器上的独立性能（以毫秒为单位），而不是常见的 CPU 时钟周期。我们观察到，Dilithium 和 Rainbow 在提供 128 位和 192 位安全性时的性能具有竞争力，与传统算法（如 RSA3072 和 ECDSA384）相当。Falcon 在验证操作方面表现优异，但签名速度较慢。可以看出，只有 Falcon 在云端机器上的运行速度较慢。我们将这一现象归因于 Falcon 使用浮点运算，而这种操作在其他后量子签名方案中并不存在。MQDSS 和 Picnic 的速度较慢，而 SPHINCS+的速度显著较低，尤其是在签名操作方面。我们的签名和验证性能测量结果与 SUPERCOP 基准测试[24][25]的结果总体一致。

表 1 不同签名算法的证书尺寸

Signature Algorithm	Cert Chain Size (KB) One ICA	Cert Chain Size (KB) Two ICAs	Certificate Verify Size (KB)
RSA 3072	1.63	2.44	0.38
ECDSA 384	1.34	2.15	0.05
Dilithium II	6.90	10.42	2.04

Falcon 512	3.54	5.37	0.69
MQDSS 48	42.24	63.42	20.85
Picnic L1FS	66.20	99.57	30.03
SPHINCS+ SHA256-128f-si	34.46	51.74	16.98
Rainbow Ia	116.86	175.35	0.06
Dilithium IV	10.70	16.11	3.37
Falcon 1024	6.56	9.89	1.33

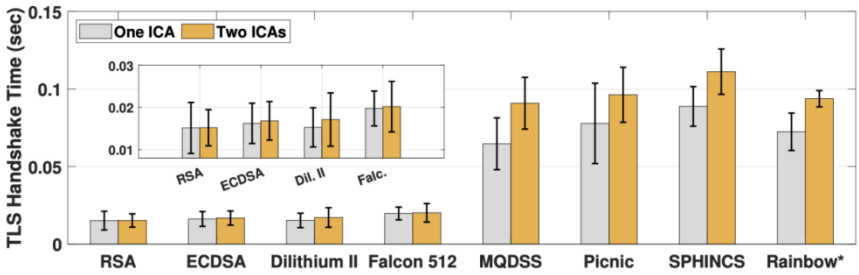


图 3 握手时延对比

2.3.2. 混合证书链设计及实现

[26]提出并研究了一种针对网络协议传输层安全（TLS）实现后量子（PQ）认证的迁移策略。我们的策略基于“混合证书链”的概念，即在同一证书链中使用不同的签名算法。为了证明该迁移策略的可行性，首先将经过充分研究且广受信任的基于哈希的签名方案（SPHINCS+ 和 XMSS）与椭圆曲线密码学（ECC）结合使用，随后与基于格的后量子密码签名方案（CRYSTALS-Dilithium 和 Falcon）结合。此外，将基于混合证书链的认证与基于格的密钥封装机制（KEM）CRYSTALS-Kyber 结合使用，作为后量子 KEM 的代表，评估一种完全后量子化且实现双向认证的 TLS 1.3 握手协议。

其研究结果表明，仅在根证书颁发机构（CA）层级使用基于哈希的签名方案的混合证书链，尽管通信数据量有所增加，但仍能实现可接受的连接建立时间。通过分析客户端和服务器程序的代码大小和峰值内存使用情况，进一步证明了这一迁移策略即使对于嵌入式设备也是适用的。

表 2 和图 4 中显示了本地实验的结果。控制组中的 EDS-EDH 和 Dil-Kyb 组合表现最佳，前者的中位数 TTFB（首字节时间）为 29.7 毫秒，后者为 30.0 毫秒。然而，将基于哈希的签名方案与 ECDSA 结合使用（作为我们迁移策略的第一步）表现出了非常有前景的结果。与传统控制组合 EDS-EDH 相比，中位数

TTFB 的增加保持在可接受范围内，尤其是在以下两个组合中：SPs+EDS-Kyb（增加 4.64 毫秒）和 XMS+EDS-Kyb（增加 5.57 毫秒）。

考虑到策略的最终阶段，观察到中位数 TTFB 的增加同样很小。在这种情况下，性能最快的两种仅使用 PQC 的混合证书链是：XMS+Dil-Kyb（增加 4.69 毫秒）和 SPs+Dil-Kyb（增加 4.41 毫秒），后者也是所有评估的混合证书链中增加幅度最小的。

此外，仅在根 CA 层使用 SPHINCS+ 的方法相比控制组中在整个链中都使用 SPHINCS+ 的证书链显示了显著的改进。具体而言，在 SPf-Kyb 的情况下，TTFB 平均减少了 79.9%；在 SPs-Kyb 的情况下，减少了 99.0%。纯 SPHINCS+ 设置中较慢的握手时间主要是由于两种特定的 SPHINCS+ 签名方案（参见表 2）较慢的签名操作，特别是在 SPs-Kyb 的情况下，实验中位数 TTFB 达到了 4.48 秒。

表 2 混合迁移算法路线

Signature Scheme	Root CA	Interm. CA	End Entity	KEX	Notation
Control	ECDSA	ECDSA	ECDSA	ECDHE	EDS-EDH
	Dilithium	Dilithium	Dilithium	Kyber	Dil-Kyb
	Falcon	Falcon	Falcon	Kyber	Fal-Kyb
	XMSS	XMSS	XMSS	Kyber	XMS-Kyb
	SPHINCS$^+$-f	SPHINCS$^+$-f	SPHINCS$^+$-f	Kyber	SPf-Kyb
	SPHINCS$^+$-s	SPHINCS$^+$-s	SPHINCS$^+$-s	Kyber	SPs-Kyb
Mixed Certificate Chain	XMSS	ECDSA	ECDSA	Kyber	XMS+EDS-Kyb
	XMSS	Dilithium	Dilithium	Kyber	XMS+Dil-Kyb
	XMSS	Falcon	Falcon	Kyber	XMS+Fal-Kyb
	SPHINCS$^+$-f	ECDSA	ECDSA	Kyber	SPf+EDS-Kyb
	SPHINCS$^+$-f	Dilithium	Dilithium	Kyber	SPf+Dil-Kyb
	SPHINCS$^+$-f	Falcon	Falcon	Kyber	SPf+Fal-Kyb
	SPHINCS$^+$-s	ECDSA	ECDSA	Kyber	SPs+EDSKyb
	SPHINCS$^+$-s	Dilithium	Dilithium	Kyber	SPs+DilKyb
	SPHINCS$^+$-s	Falcon	Falcon	Kyber	SPs+Fal-Kyb

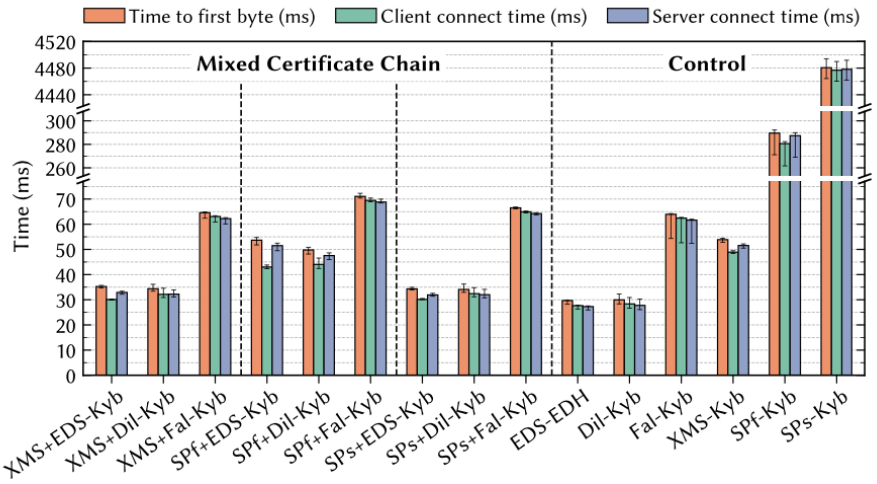


图 4 迁移后的结果

2.3.3. 多种证书混合模式设计研究

[27]中提出了多种混合证书链的模式，进行研究分析不同混合模式下的安全性及效率。目前混合认证方式中在混合签名方案中研究最多，以下主要研究混合签名方案。在 2023 年，Nina 等人提出并实现了证明可组合性（Proof Composability）、SNS 和同时验证（Simultaneous Verification）（下文将其组合称为混杂）。他们给出了几种基于 Fiat-Shamir 签名方案（如 Dilithium）或 Falcon 与 RSA 或 DSA 的组合。并观察到，有许多签名混合目标，其中一些通过并行签名或串联方法无法实现。这些目标包括证明可组合性（即后量子混合签名的安全性可以轻松链接到组件算法）、弱可分离性、强可分离性、向后兼容性、混合通用性（即一旦被证明是安全的，可以用不同算法实例化的混合组合）和同时验证。其展示了抗量子混合认证方式在网络协议中的可行性。

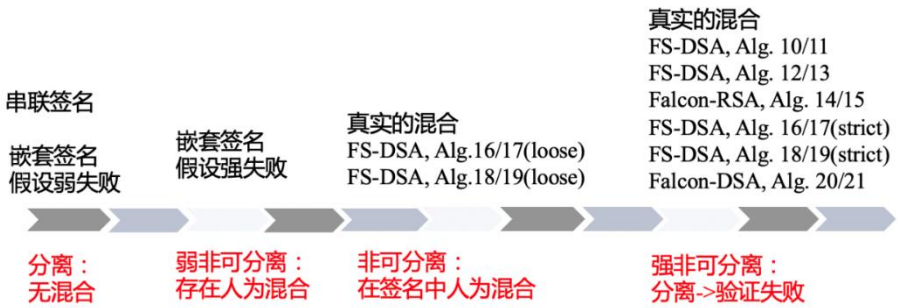


图 5（非）分离混合签名方案

非可分离性表示可分离性程度。嵌套签名（其中消息由一个组件算法签名，然后消息-签名组合由第二个组件算法签名）属于这一类别，这取决于内层或外层签名是否被“剥离”。在弱混合失败的情况下，如果外层签名被剥离，混合签名的任何痕迹可能都不会保留（取决于各种实现细节）。

在弱非可分离性下，如果一个混合签名的复合签名被移除，混合的痕迹将会保留。这使得验证者能够检测到组成签名从混合签名中被剥离。例如，在一个串联的 ECDSA-Dilithium 混合中，需要一个经过认证的 ECDSA-Dilithium 混合公钥组合，ECDSA 组件公钥可以用于验证仅包含 ECDSA 组件签名。在这种情况下，检测取决于认证混合公钥的实现要求的“痕迹”。这说明混合签名是否可分离或（弱）不可分离也可能取决于协议实现的层面，即超出算法级别的安全方面，例如 EUF-CMA 之外的标准定义。嵌套签名提供了另一个例子，在强混合失败下，验证者可能被欺骗，将一个新消息解释为消息/内部签名组合，并仅验证外部签名。在这种情况下，内部签名标签也是一个“痕迹”。

在该规模上的第三个概念是非可分离性概念，其中可分离性检测依赖于签名本身的痕迹。与弱非可分离性不同，其中痕迹可能在实际消息、证书或其他非签名组件中，这个概念更密切地与传统的算法安全概念（如 EUF-CMA）相关，其中安全性取决于签名算法的内部结构和验证过程。在这种类型中，验证者能够在算法级别的验证过程中检测到痕迹。

在最强的非可分离性概念的方案中，当其中一个组成签名缺失时，验证不仅失败，而且如果验证者无法验证两个签名，验证也会失败。这种非可分离性构造最接近传统的数字签名，其中假设验证者确实验证了一个签名，结果要么是对完整签名的肯定验证，要么是如果签名无效则失败。对于混合签名，一个“完整签名”意味着两个组件算法的存在，因此最强非可分离性概念对验证采取了全有或全无的方式。因此，经历了任何分离攻击的更可分离的签名都无法实现混合验证。

2.3.4. Google 混合认证方式

Google 团队实现了开源安全密钥固件 OpenSK 的一部分，发布了第一个量子弹性 FIDO2 安全密钥实现。此开源硬件优化实现使用了一种新颖的 ECC/Dilithium 混合签名方案，该方案受益于 ECC 对标准攻击的安全性和 Dilithium 对量子攻击的弹性。

[28]提出的实施方案依赖于一种混合方法，该方法结合了经典 ECDSA 签名算法和抗量子签名算法 Dilithium。依赖混合签名至关重要，因为 Dilithium 和其他最近标准化的抗量子算法的安全性尚未经受住时间的考验，这对于密钥安全尤其有必要，因为大多数密钥都无法升级。在技术方面，一个巨大的挑战是创建一个足够小的 Dilithium 实现，以便在安全密钥受限的硬件上运行。通过优化，能够开发一个 Rust 内存优化的实现，它只需要 20 KB 的内存。

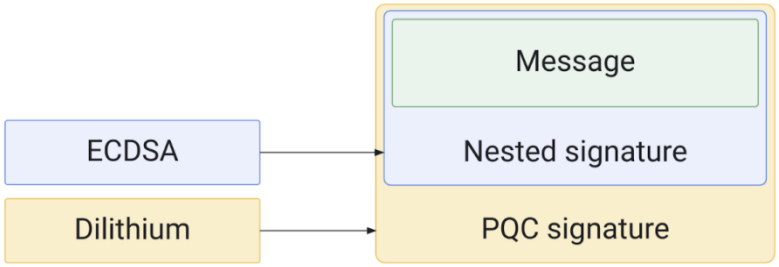


图 6 混合签名：经典和 PQC 方案的强嵌套

2.4. 国内外抗量子证书研究现状

2018 年，Panos Kampanakis 等人提供一种证书混合机制，即在一个传统算法的数字证书中，采用 X509 V3 扩展表达抗量子密码算法的算法标识、公钥以及 CA（Certificate Authority，证书授权机构）对内容的签名。IETF 在其发布的抗量子相关标准草案中提到，在抗量子密码算法被进行更多更充分的安全分析审查和实现错误审查之前，出于安全性和兼容性考虑，建议过渡阶段使用多种混合方案实现抗量子密码迁移。2022 年以来，IETF 针对证书的迁移陆续发布多个标准草案，其中给出了三种方式：1）多证书，即用户拥有两张证书，分别是常规证书和抗量子密码证书，通过扩展字段将扩展证书和抗量子密码证书绑定。2）混合非复合证书，即在一张证书上提供常规公钥和抗量子密码公钥，CA 同时完成常规签名和抗量子密码签名。该方式可实现后向兼容，即在未来全面实现了抗量子密码迁移后，CA 也不用向用户颁发新的抗量子密码证书。3）混合复合证书，即将多个公钥和私钥值编码到现有公钥和私钥字段，如使用 Dilithium-RSA 等复合算法。该方式只需对密码算法进行调整，避免了对协议进行改造。不同混合方式的数字证书针对不同的应用具有不同优缺点，需要基于特定应用场景进行具体分

析和适应性调整。2023 年，北京数字认证股份有限公司基于 Dilithium 算法[29]做了抗量子证书签发试验，成功签发了基于 Dilithium 的抗量子证书。

尽管目前已有相关混合数字证书格式方案被提出，但其与不同密码应用系统的适配性仍需进一步研究。如若混合证书格式设计导致尺寸过大，将直接影响密码安全协议的稳定性。此外，数字认证体系较为复杂，抗量子数字认证体系构建不仅涉及证书格式的设计，还包括数字证书的签发与状态查询这一基础功能，由于不同技术路线抗量子签名算法性能及尺寸具有较大差异，为满足不同密码应用对证书的需求，抗量子数字认证体系需要能够支持不同技术路线抗量子密码算法证书的签发与状态查询。

结合上述迁移中出现的问题，本课题进行的方案设计更关注于证书格式的设设计、证书生成工具的安全性和其抗量子签名算法的性能和尺寸问题。分析密码应用对数字证书的签发速度、证书大小的要求，进而分析数字证书对密码算法参数的要求，选择适配的密码算法与算法参数。根据选定的密码算法和算法参数结合与现有产品和未来业务的兼容需求，研究设计适配的抗量子/混合证书格式；开发支持多种证书格式的证书签发工具，支持至少一种密码算法以及多种算法参数，用于支持密码协议的试验验证。

在向抗量子密码学（PQC）迁移过程中，X.509 证书发挥着传递和交换公钥的关键作用，并广泛应用于 TLS/SSL[30]、QUIC[31]、S/MIME 和 IPsec[32]等协议。为了适应 PQC 需求，已提出多种改进的 X.509 证书格式。其中，一些格式为纯 PQC 设计，仅包含 PQC 公钥并由 PQC 签名算法签名；另一些格式则为混合设计，同时携带传统公钥和 PQC 公钥，并通过传统和 PQC 签名算法共同签名。这些改进为 X.509 证书在抗量子密码学环境中的应用奠定了基础。

以下是混合证书的一些改进现状[33]：

1. 混合连接格式。NIST 提出通过将传统加密对象与抗量子加密对象（如：公钥、签名）简单连接起来而无需更改 ASN.1 树结构，如**错误!未找到引用源。**所示，实现了传统与抗量子技术的共存。在验签环节需要对公钥域和签名域同时进行抗量子和经典的公钥和签名验证。



图 7 混合连接模式证书结构

2. 混合绑定格式。IETF（网络工程任务组）提出使用两个分开的证书，一个基于传统加密技术，另一个完全采用 PQC 构建，CA 在生成证书时，会在经典证书和抗量子证书之间嵌入相互指向的扩展字段，从而建立双向链接。这些扩展字段携带关联证书的标识信息，确保验证者在使用其中一张证书时能够快速找到并验证另一张证书。RelatedCertificate 扩展包含关联证书的唯一标识信息，包括关联证书的完整哈希值（使用与证书签名相同的哈希算法计算）以及其他必要的元信息（如证书的颁发者和序列号）。在验证阶段中，验证者在收到包含 RelatedCertificate 扩展的证书时，会根据扩展字段中提供的信息计算关联证书的哈希值，并验证其与扩展字段中的哈希值是否匹配。如果匹配，验证者可确定两个证书确实属于同一实体，并可以在认证或加密通信中灵活使用双证书。

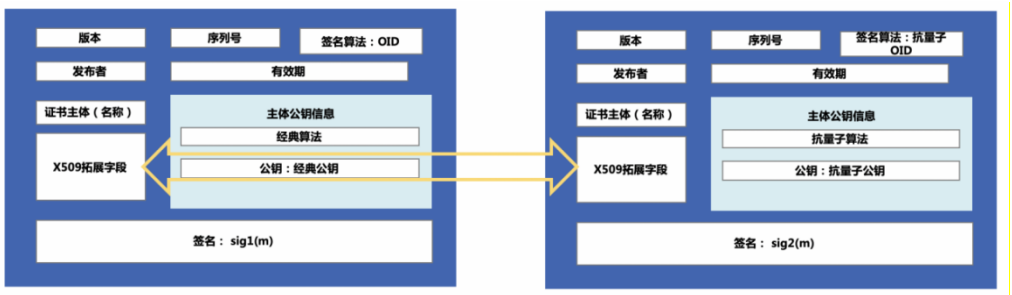


图 8 混合绑定模式证书结构

3. 混合复合格式。IETF 提出“混合连接格式”的改进版，使用 ASN.1 编码明确区分传统对象和抗量子对象，提高了结构清晰度和解析效率。通过使用明确的字段划分、自描述性标识和模块化封装，实现了传统对象和抗量子对象的清晰区分，不仅提高了数据结构的透明性，还增强了解析效率和未来扩展能力。这种设

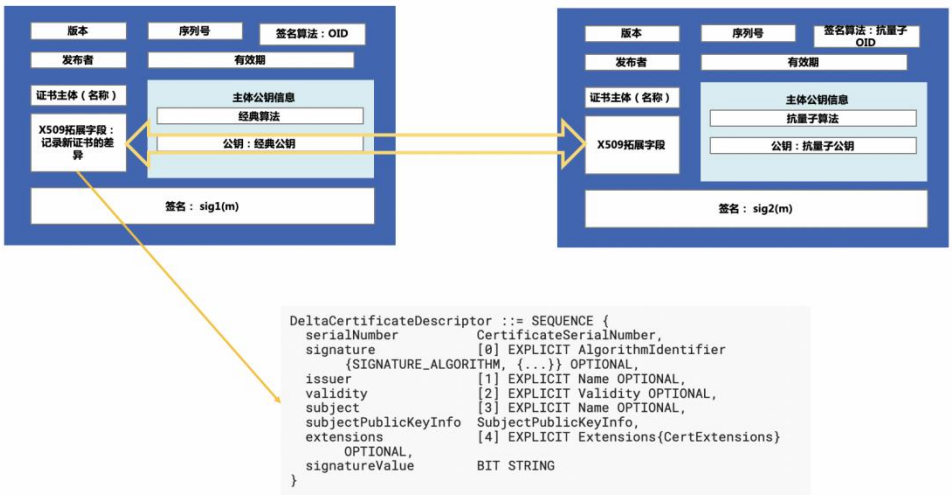
计在支持传统与抗量子加密技术共存的同时，为系统平滑过渡到抗量子环境提供了强有力的技术保障。

4. 混合使用扩展。将抗量子对象存储在证书的扩展字段中，使证书在保持传统 X.509 格式的同时，加入抗量子功能。其定义一个新的扩展字段。



图 9 混合使用拓展证书格式

5. 混合差分扩展。通过单一扩展字段编码两个证书间的差异，其中一个证书作为基础，另一个作为差分，以实现传统和抗量子技术的整合。这些格式为证书的使用者和管理者在抗量子环境中提供了更多选择与灵活性，以对抗未来量子计算机的潜在威胁。




```
DeltaCertificateDescriptor ::= SEQUENCE {
  serialNumber          CertificateSerialNumber,
  signature             [0] EXPLICIT AlgorithmIdentifier
       {SIGNATURE_ALGORITHM, {...}} OPTIONAL,
  issuer                [1] EXPLICIT Name OPTIONAL,
  validity              [2] EXPLICIT Validity OPTIONAL,
  subject               [3] EXPLICIT Name OPTIONAL,
  subjectPublicKeyInfo  SubjectPublicKeyInfo,
  extensions            [4] EXPLICIT Extensions{CertExtensions}
       OPTIONAL,
  signatureValue        BIT STRING
}
```

图 10 混合差分拓展证书结构

3. 数字证书基础介绍

随着量子计算技术的迅猛发展，传统的密码学算法面临着前所未有的安全挑战。经典算法如 RSA、DSA 和 ECC 在量子计算的威胁下，其安全性受到质疑。例如，Shor 算法能够在多项式时间内有效破解 RSA 和 ECC，这使得基于这些算法的加密通信和身份验证机制变得脆弱。因此，全球信息安全领域迫切需要新的解决方案，以应对未来可能出现的量子攻击。

在这一背景下，抗量子密码学逐渐成为研究的热点。抗量子密码学基于数学上更为复杂的结构，例如格、编码理论和多变量多项式，旨在提供比传统算法更强的安全性。其中，Kyber 和 Dilithium 等后量子算法已被提议作为新一代密码学标准的候选者。这些算法不仅在理论上被证明具备抗量子安全性，还在性能和效率上做出了优化，适应了现代网络环境的需求。

本研究旨在设计和实现基于抗量子密码算法的 X.509 证书方案，探索其在网络安全通信中的应用，尤其是在 TLS 协议的安全性增强方面。通过整合抗量子算法与现有的公共密钥基础设施（PKI），我们期望为网络安全提供一个可持续且有效的解决方案，以确保在未来网络环境中的数据机密性和完整性。
学术界和产业界对认证密钥交换协议进行了大量研究，下面介绍混合证书的研究现状，主要包括经典证书认证、抗量子证书认证、混合证书认证以及经典/抗量子证书在实际网络协议中的应用现状。

随着信息技术的迅猛发展，数字证书作为实现身份验证和数据安全的核心机制，其标准化工作逐渐成为研究的重点。目前，X.509 是最广泛采用的数字证书标准之一，提供了结构化的方式来定义证书的格式和内容，广泛应用于 TLS/SSL 等安全协议中。X.509 证书不仅包含了持有者的公共密钥、有效期、颁发机构的信息，还包含了使用的签名算法及其相关参数。

在 X.509 标准中，经典签名算法如 RSA[34]、DSA[30]和 ECDSA[32]是最常用的算法。RSA 算法以其成熟和广泛的应用基础，成为 X.509 证书的主要签名算法之一。其安全性依赖于大数分解的困难性，尽管在过去几十年中，RSA 在身份验证和数据加密方面得到了广泛应用，但其较长的密钥长度和计算开销限制了其在资源受限环境中的使用。DSA 作为另一种被广泛采用的签名算法，其设计

初衷在于提高签名生成和验证的速度，并且被纳入美国国家标准的数字签名标准（DSS）。然而，DSA 的安全性同样受到密钥长度和量子计算的威胁，限制了其在未来应用中的前景。相较之下，ECDSA 利用椭圆曲线理论，在提供相同安全级别的情况下实现了更短的密钥长度，提升了签名的效率。随着移动设备和物联网的兴起，ECDSA 因其高效性和安全性而逐渐成为 X.509 证书中首选的签名算法。然而，ECDSA 也并非没有缺陷，其安全性同样受到量子计算的威胁，使得研究者们开始关注抗量子签名算法的可能性。在应对这些挑战的过程中，研究者们探索将抗量子算法与现有标准结合的可能性，推动了对基于 X.509 的抗量子证书的研究。这些新兴的证书方案，如使用基于格的签名算法或哈希基算法，旨在在不牺牲性能的前提下，增强数字证书的安全性。

基于经典签名算法的证书标准如 X.509 在信息安全领域中仍然占据重要地位，但面对量子计算等新兴威胁，未来的研究方向将集中于如何将抗量子算法有效地融入现有标准中，以确保数字证书在快速变化的网络环境中的可靠性和安全性。

3.1. 国密算法与证书格式

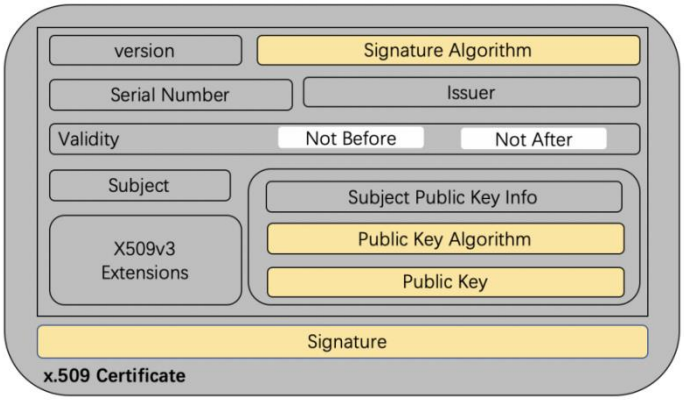


图 11 X.509 证书格式

数字证书是 PKI 体系中绑定实体身份与公钥的载体，国际通用的 X.509 v3 数字证书与我国国家标准 GB/T 20518-2018《信息安全技术 公钥基础设施 数字证书格式》整体 ASN.1 数据框架、基础字段、DER 编码及 PEM 封装规则高度相似，因此下文以 X.509 v3 为例介绍证书结构。

X.509 与 GB/T 20518-2018 标准证书二者主要存在以下差异：X.509 为国际标准，无强制密码算法约束，兼容 RSA、NIST ECC 等各类国际公钥算法，仅定义通用国际扩展，适配全球互联网跨境通信场景，无地域行业强制合规要求，配

套 RFC 5280 等国际规范，而 GB/T 20518-2018 是适配我国商用密码体系的本地化标准，强制规范 SM2、SM3 等国密算法的编码与使用。此外，X.509 采用单证书体系，一张证书内的同一对公钥同时承担签名、加密两种功能；GB/T 20518-2018 规定国密采用双证书体系，分为独立的签名证书、加密证书。

3.1.1. 版本 (Version)

X.509 证书的版本标识了证书结构和功能支持的范围，目前定义了三个版本：v1、v2 和 v3。v1 是基础版本，仅提供基本的身份验证信息；v2 引入了少量改进，例如支持证书吊销列表 (CRL) 分发点，但实际产业部署中使用较少；v3 是当前互联网体系内最广泛使用的版本，原生支持可自定义的灵活扩展字段，涵盖密钥用途、主题替代名称 (SAN) 等拓展项，能够兼容国产商用密码证书扩展规范，为 SM2 国密证书、后量子密码证书等定制化场景提供拓展能力支撑，是 SM2 证书落地的核心载体版本。

基于国密 GM/T 0015、GM/T 0102 标准的 SM2 商用密码证书均基于 X.509 v3 框架构建，不会采用功能受限的 v1、v2 版本：SM2 证书将国密专用公钥标识、加密用途约束、国密算法策略、双证书区分标识（签名证书 / 加密证书）等专属属性封装在 v3 独立扩展字段中，完整保留 X.509 原生基础字段的标准语义。v3 版本的扩展隔离特性保证 SM2 国密算法配套参数、算法约束扩展项与传统 RSA/ECC 字段互不干扰，既可以兼容传统 PKI 解析架构，又能合规承载 SM2 签名公钥、加密公钥及国密签发签名，成为政务、金融等国产化场景下 SM2 证书规模化部署的必要前提。整体而言，版本字段直接决定证书可用特性集、算法拓展能力，同时影响 SM2 国密体系、传统密码体系与 TLS 等现代通信协议的双向兼容性。

3.1.2. 序列号 (Serial Number)

序列号是由证书颁发机构 (CA) 分配的唯一标识符，用于区分其颁发的每一张证书。该字段的唯一性确保了证书的可跟踪性和便捷管理。例如，当证书因私钥泄露或其他原因被吊销时，其序列号将记录在证书吊销列表 (CRL) 中或通过在线状态协议 (OCSP) 提供状态查询，从而帮助客户端快速识别无效证书。

3.1.3. 签名算法标识符 (Signature Algorithm Identifier)

签名算法标识符指定了用于对证书内容进行数字签名的加密算法，包括哈希算法和签名方法。常用的签名算法包括 RSA（如 SHA256-RSA）、ECDSA 和 EdDSA 等。这个字段直接影响证书的安全性和兼容性，现代算法通常结合较强的哈希函数（如 SHA-256）来确保抵抗潜在的攻击。签名算法还决定了证书在不同应用场景下的适用性。

3.1.4. 签发人名称 (Issuer Name)

签发人名称标识了颁发证书的机构，通常是证书颁发机构 (CA)。该名称以 X.500 格式表示，例如 “CN=Root CA, O=Example Organization, C=US”。在信任链中，签发人名称必须与上级证书的“主体名称”匹配，这确保了信任链的完整性。对于根 CA 证书，签发人名称与主体名称相同，因为它是自签名证书。

3.1.5. 有效期 (Validity Period)

证书的有效期定义了其生命周期，包括起始时间（Not Before）和终止时间（Not After）。在有效期内，证书被视为可信；超过终止时间后，它将被自动作废。有效期的长度通常根据应用场景和安全性需求而定，例如短期证书更安全但管理成本更高，而长期证书适用于根 CA 或内部应用场景。现代浏览器通常不接受有效期超过 398 天的 TLS 证书，以降低私钥泄露的风险。

3.1.6. 主体名称 (Subject Name)

主体名称标识了证书持有者，通常以 X.500 格式的区分名 (DN) 表示，例如 “CN=www.example.com, O=Example Organization, C=US”。这个字段用于明确持有者的身份，可以是个人、组织或设备名称。在现代应用中，“主题替代名称” (SAN) 扩展字段常用来补充或替代主体名称，支持多个域名或 IP 地址，从而提高证书的灵活性和适用范围。

3.1.7. 主体公钥信息 (Subject Public Key Info)

主体公钥信息包含了证书持有者的公钥及其相关算法参数，用于加密数据或验证数字签名。这个字段包括公钥值、算法类型（如 RSA 或 ECDSA）以及密钥参数（如 RSA 的密钥长度或椭圆曲线名称）。客户端使用该信息与私钥配合完成安全通信，是整个 PKI 系统中最核心的部分之一。

3.1.8. 扩展字段 (Extensions)

扩展字段是 X.509 v3 的一大特色，用于提供灵活的附加信息，支持证书在不同场景下的应用。例如，基本约束字段指明证书是否为 CA 证书；密钥用途字段限制公钥的功能范围，如签名或加密；主题替代名称 (SAN) 扩展字段允许一个证书同时绑定多个域名或 IP 地址。国产 SM2 商用密码证书同样依托 X.509 v3 扩展字段实现国密定制化适配，通过专属扩展项区分 SM2 签名证书与 SM2 加密证书、标注 GM/T 系列国密算法标识、限定国密密钥合规使用范围，满足政务、金融等国产化场景的安全合规要求。这些扩展字段极大地增强了证书的功能性和适用性，在传统公钥体系与国密商用密码体系中均得到广泛部署。

在基于上述证书结构体系下，若直接整体替换 X.509 证书的签名算法、公钥和签名域，需全面评估技术、安全和兼容性影响，该约束在 SM2 国密证书改造与跨算法迁移场景下尤为突出。在替换签名算法时，通用场景可选择符合现代安全标准的算法（如 SHA-256 或 ECDSA），国产化场景则需适配 SM2withSM3 国密签名套件，既要验证国密算法与存量终端、国密改造网关的兼容性，也要完整更新证书链以保障链路完整性；替换公钥环节中，若切换为 SM2 公钥，需合规生成符合 GM/T 0006 规范的 SM2 公私钥对，同步完成国密密码机、业务系统密钥配置同步，杜绝密钥泄露与密钥生命周期超限风险；对于签名域的变更，无论是传统算法替换还是向 SM2 国密算法迁移，均需要按需更新主体名称或主题替代名称 (SAN) 字段以适配新身份标识，重新签发合规证书，保障普通客户端、国密适配客户端与全量依赖业务服务均可完成合法证书校验。

3.2. 数字证书的生命周期

如图 12 数字证书的全生命周期是保障信息系统安全的核心流程，涵盖了密钥生成、证书签发、证书存储与管理、证书更新及撤销的多个阶段。密钥对的生成是数字证书生命周期的起点，通常在一个安全环境中完成，以确保密钥的机密性和抗攻击能力。接下来，认证机构（CA）根据申请者提供的资料验证其公钥的合法性，并为其签发数字证书，确保公钥与特定身份的绑定。生成的证书应当被妥善存储，通常使用软硬件结合的加密存储设备，如加密模块或密钥管理系统，以防止未经授权的访问。证书的分发和使用依赖于证书服务器，证书服务器支持在线验证（如 OCSP）和离线验证（如 CRL），以确保信任链的完整性。在密钥更新或算法迁移的过程中，证书在到期或出现安全风险时需及时更新，以确保系

统的连续性和兼容性。最后，当证书失效或被认为不再可信时，通过撤销机制（例如发布证书吊销列表或启用在线状态检查）来标记无效证书，防止潜在的安全隐患，以实现持续有效的身份认证和数据保护。

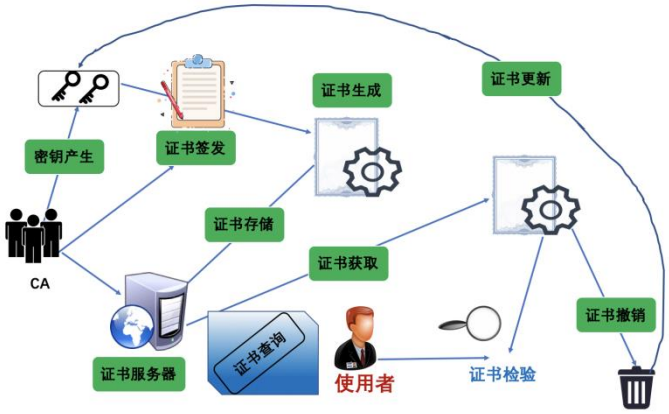


图 12 证书全生命周期

3.2.1. 证书的生命

数字证书在由认证机构（CA）签发并生效后，依赖方即可基于该证书进行多种操作。数字证书将公钥与证书持有者的身份信息绑定，是公钥基础设施（PKI）系统中用于发布公钥的核心数据元素。证书本身只是按照特定格式定义的数据文件，而证书的操作则指对证书内数据内容进行密码学处理，从而实现其多种功能。例如，签名验证操作是对证书中的签名结果进行解密运算并比对，从而验证证书的有效性。

数字证书的各类操作是保证 PKI 系统正常运行的关键。本报告将数字证书的操作分为五个主要方面：证书的生成操作（包括密钥生成、申请提交、审核检查和证书签发）；证书的使用操作（包括证书的获取和验证使用）；证书的撤销；证书的更新；以及证书的归档。

当某个实体希望加入 PKI 系统并成为完整的 PKI 实体时，首先需要由认证机构（CA）为其签发数字证书，作为其在 PKI 系统中的“身份标识”。这个过程包括两个主要步骤：提交证书申请材料和经过 CA（有时由注册机构 RA）审核后由 CA 签发证书。

证书申请材料通常包括两部分内容：第一部分是身份证明材料，包含申请者的个人身份信息及 CA 要求的其他资格审核资料；第二部分是申请者希望绑定至证书的公钥（在某些情况下，密钥材料可能由 CA 或其他第三方提供）。

CA 或 RA 将对提交的材料进行核实和审核，确保其真实性和合法性。一旦审核通过，CA 将根据相关政策签发相应的数字证书。

3.2.2. 证书的使用

3.2.2.1. 根证书的获取

根 CA 证书由于是一种自签名的证书，订户在拿到一份根 CA 的证书后，无法通过 PKI 系统的技术手段对其进行验证。根证书的获取只能采用带外方式。根 CA 证书的获取还应该注意在选择带外方式和证书安装时的安全性，特别是需要防止替换或者篡改等攻击，因此，带外方式是一个非常安全的信道。

3.2.2.2. 订户证书的获取

订户证书的获取一般不用通过带外方式。依赖方可以从 CA 的数字证书资料库获取证书，也可以是证书的持有者通过其它途经发送给依赖方，且不用担心传输信道的安全性。CA 的数字证书资料库也应该提供多种发布或者获取的方式，比如 FTP、WEB 以及 X.500 目录等。发布证书的 PKI 组件应该提供良好的查询接口，后台程序应该支持灵活多样的检索方式，比如基于证书主体名称的检索等等。

3.2.2.3. 验证使用

数字证书存在的目的是能被用来验证该公钥确实属于证书持有者。

PKI 系统的依赖方通过验证证书确信这个绑定关系后，一般是为了两种方式的使用：利用公钥直接或者间接（加密对称密钥）加密某些仅希望证书持有者阅读的信息。利用公钥验证证书持有者的签名。对证书持有者的签名验证成功后，依赖方又可以继而得到如下两个信息：

第一，确实是该个人信息的拥有者签署了这个签名。

第二，该个人信息的拥有者将不能否认他的签名。

对数字证书的验证在需要多个操作。比如一个由非根 CA 签发的数字证书的验证就会涉及到下面的步骤：

首先审查持有者的证书是否依然有效。这包括查看证书是否已过失效日期，并且查看证书的撤销情况。

获取该证书的签发者的数字证书来验证该证书上的签名。拿到签发者的数字证书后，同样需要检查证书是否已过失效日期，并且查看证书的撤销情况。该签发者数字证书上会有一个根 CA 的签名。

获取根 CA 证书，同样查看失效日期和证书撤销情况。由于根 CA 是自签名证书，那么不需要其它的证书来验证根证书上的签名。

用根 CA 的公钥来验证签发者 CA 的证书上的签名，确信了签发者 CA 的公钥。

用签发者 CA 的公钥来验证证书上的签名，因此可确知证书持有者的公钥。

注意，查看证书撤销列表（Certificate Revocation List，简称 CRL）时，撤销列表也会有根 CA 或者下级 CA 的签名，验证撤销列表的证书的过程也类似于上述 5 步所描述的过程。如图 13 所示，依赖方对 Bob 的数字证书的验证流程：

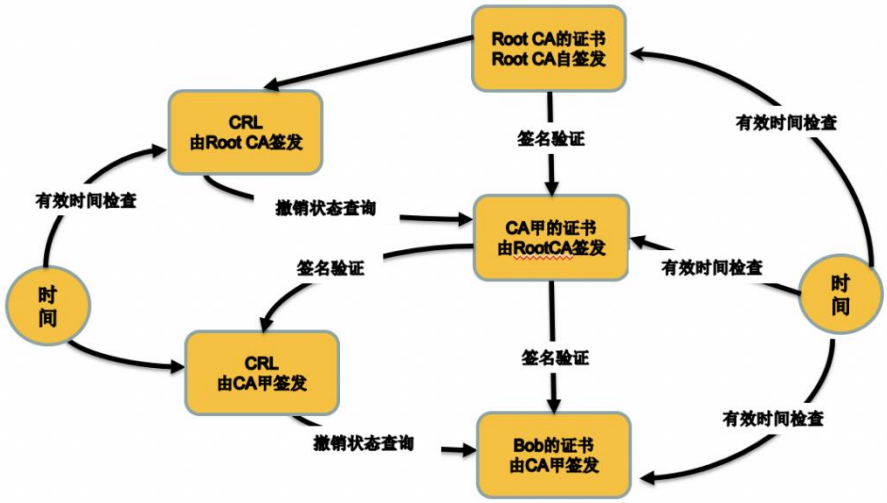


图 13 Bob 证书的验证

3.2.2.4. 证书存储

CA 给订户签发了数字证书后会给订户发送生成的证书，订户将证书存储在本地以便日后使用。主要的使用形式就是直接发送给其它实体，供其鉴别自己的身份。一般来说，本地系统会存储有很多其它实体的证书，订户甚至都不知道它们的存在却在使用它们提供的服务。这些证书可能来自不同的证书颁发机构，而来自同一个颁发机构的证书也可能有很多份，这些证书具有不同的作用，订户证书、下级 CA 证书以及根 CA 证书都可能存储在本地系统中。

3.2.2.5. 证书撤销

CA 一般采用证书撤销列表的方式将被撤销的证书告知其他订户，CRL 中列举着所有在有效期内但被撤销的数字证书。

证书撤销的流程如下：

订户或者其上级单位向 RA 提出撤销请求；

RA 审查撤销请求；

审查通过后，RA 将撤销请求发送给 CA 或者 CRL 签发机构；

CA 或者 CRL 签发机构修改证书状态，并签发新的 CRL。

当该数字证书被放入 CRL 后，数字证书则被认为失效。注意，失效并不意味着无法被使用。如果窃取到甲的私钥的乙用甲的私钥签名了一份文件发送给丙，并附上甲的证书，如果丙忽视了对 CRL 的查看，丙就依然会用甲的证书成功的验证这份非法的签名，并会认为甲确实对这份文件签名。

为了让各依赖方能够容易、及时、正确地了解证书的撤销信息。CA 一般会初始化和维持一个 CRL，罗列着已经被撤销的数字证书。CRL 的发布方式一般是通过网络，包括 Web、FTP 等，在数字证书的扩展里一般有存放 CRL 的地址。除此之外，本地一般会通过一个 CRL 缓存机制来缓存着一份最新的 CRL。

然而如果 CA 非常频繁地更新 CRL，那么 CA 运营的成本将提高，但是可以把作废证书可能造成的损失降低。反之，降低 CRL 的更新频率，CA 在节省这方面成本的同时，也加大了作废证书由于没有及时发布而使得依赖方蒙受损失的风险。CA 应该选择合适的证书撤销列表发布策略来达到最优化的成本和风险控制。上面所提到的 CRL 缓存时间和 CA 的 CRL 更新周期应该是一致，使得本地缓存的 CRL 是当前最新的 CRL。

常用的证书撤销状态发布方式还有在线证书状态协议（Online Certificate Status Protocol，简称为 OCSP），该协议为依赖方提供对证书状态的实时在线查询。依赖方不用检查证书撤销列表而是向证书状态查询服务器在线询问某证书的状态。但是 OCSP 服务器必须对查询的结果进行数字签名，而签名是消耗计算资源的操作，所以对 OCSP 服务器的要求很高。由于查询者可以是任何实体，没有办法进行身份验证，所以 OCSP 服务器的这种在线的响应方式很可能会遭到 DOS/DDOS 攻击。

证书撤销树（Certificate Revocation Tree，简称为 CRT）是一个比较折中的方案。CRT 是基于二叉 HASH 树的对传统 CRL 的改造，它基本上拥有了 CRL 和 OCSP 的优势，且可以快速地返回查询结果。

3.2.2.6. 证书的更新

订户在数字证书即将过期的时候就需要一份新的证书。如果此时重新完成一次初始申请时的申请过程，订户需要重新收集自己的各种信息，CA 也同样需要重复劳动来审核。在订户仅仅更换有效期或更换密钥的过程不需要核查信息便可以尽快地签发新的证书，实现证书的更新。

证书更新的初衷是使 PKI 用户无间歇地持续使用 PKI 服务。因此，证书订户应当在证书到期之前向 CA 提出证书更新请求，比如在到期前的一个月或者一周就提出更新请求，以便 CA 在订户证书失效前为其更新。新证书的生效日期一般早于旧证书的失效日期，而新证书的失效日期将相应的延后。

证书的更新需要 CA 签发一份新的证书，但是此时的审核签发过程和订户第一次申请时是不同的。在需要更新时，证书订户已经在初始申请时经过了 CA 的各项审核，此时证书订户的目的仅是希望获得一个失效期更晚的证书来继续享受 PKI 系统的服务，那么 CA 完全可以不对此证书订户的身份信息进行审核。CA 此时需要生成的新证书的主要内容跟现有证书相比基本一样，甚至沿用现有的公钥，不同之处仅仅在于序列号以及生效和失效日期。而证书第一次的产生操作中，由于对申请人认知的空白，需要申请人通过各种形式提供所需的身份证明材料，且提供能够进行 POP 检查的公钥，再经过严格审查才能提供相应级别的证书。可见，证书的更新操作和证书的初始产生操作最大的不同就在于审核过程。

订户要进行证书的更新操作，需要满足以下条件：

1. 订户现持有一份尚且有效的证书。
2. 该证书绑定的公钥所对应的私钥不存在泄露等安全问题。
3. 更新请求仅是要求延后失效日期或更换一对密钥。

CA 在收到了满足以上条件的订户证书更新请求时，可以进行非常简化的审核过程（或者完全省掉审核），直接利用原有证书的各项内容，仅需修改生效日期和失效日期，然后 CA 签名发布即可。

除了延长临近到期的证书的有效期之外，在其它情况下订户也可能会向 CA 提出证书的更新以变更自己的公私密钥对。需要注意的是，这样的变更请求不能是因为旧证书绑定的公钥所对应的私钥泄露或者疑似泄露等安全原因。

订户在更新时要求更换密钥一般是以下几种情况。

比如，订户本身对密钥定期更新有自己的要求（与 CA 公司无关），那么在密钥达到更新期时，哪怕 CA 颁发的证书仍然有效且没有任何其它异常，订户也会向 CA 提供新的公钥，要求更新证书。当然，CA 公司如果对密钥也有类似的定期更换要求，到达更换期时，CA 公司也会为订户更新证书。此外，如果研究发现某种长度的密钥或者某种加密算法不再安全，订户证书恰好使用这类算法，那么订户也可提出证书的更新要求。注意，当更新操作涉及到密钥的变更时，必须重新进行 POP 检查。

CA 不应接受其它形式的证书更新请求。比如，订户可能要求更改数字证书中的某些个人信息。主要原因就是这样的变更请求直接牵涉到 CA 在对该订户第一次请求证书时的审核。订户初始申请时，CA 是以原始材料中的信息进行审核和批准。但现在订户要求修改的内容改变了原始材料中的某项，而此时修改后的结果如果放到最初的申请审核时，可能造成第一次申请材料的审核不能通过。又如，订户要求更新证书使得新证书具有更高的安全等级。因为 CA 是根据最初的申请材料和 CPS 来决定颁发某种安全等级的证书的，所以要求改变安全等级之类的更新请求也完全不符合更新的基本前提和目的。

在此对上述个人信息项更新的问题进行补充说明。数字证书里的个人信息部分，从标准上来说是有必须检查和无需检查之分的。所以如果订户在更新请求中要求修改的个人信息部分属于无需检查的部分，那么从原则上来说 CA 可以接受这类更新请求。但是实际中，无需检查的部分基本很少被 CA 所采用，也就是说绝大多数情况下都为空。

此外，有的证书订户在签名私钥被销毁后（如存放私钥的 USB KEY 设备遭到物理损坏）也会请求 CA 公司进行证书更新。由于订户此时没有了私钥（也没有机构能恢复私钥），CA 无法通过在线的技术手段来鉴别订户的身份，无法确知这个更新请求是否由该订户所发出，因此 CA 无法在线地接受更新请求。但订户可通过带外方式向 CA 证明自己的身份，比如到 CA 公司出示身份证、USB

KEY 等。通过带外的方式鉴别了订户的身份和更新请求后，CA 就可按照证书更新的流程为订户签发新证书。

3.2.2.7. 证书的归档

PKI 系统所产生过的数字证书的总量大于当前有效的数字证书的总量。如果把 CA 所颁发的数字证书看作一个集合，而且把集合中元素的个数定义为该 CA 自建立以来所颁发的所有数字证书的个数，由于更新、撤销以及新的证书签发，这个集合是不断膨胀的。证书的更新和撤销形成了一个非常活跃的动态形式，并造成了大量的无效（各种原因导致）证书。PKI 系统存储这些已经失效的数字证书，也就是证书的归档。此外，CA 所发布的 CRL 信息，在这方面跟数字证书具有类似性，CRL 信息也具有活跃的动态性，假设 CA 每天发布一次 CRL，那么每天都会产生一个作废失效的 CRL。PKI 也应当对 CRL 采取类似于数字证书一样的归档操作。

PKI 系统希望提供有延续性的验证服务，则必须通过归档来做到。如果数字证书或者 CRL 在被更替后没有进行合适的归档处理，那么 PKI 系统很可能失去对过去的一些签名提供验证服务。

试举一个简单的例子来说明。假设 CA 在 2000 年 1 月 1 日给甲签发了一个序列号为 12345 的数字证书，该证书的到期日为 2010 年 1 月 1 日。甲在 2005 年 1 月 1 日用此证书的私钥签名了一份文件 A。甲在 2006 年 1 月 1 日发现私钥泄露，当日对此证书申请撤销，CA 在当日批准并加入当日的 CRL。此后，甲向 CA 重新申请证书，CA 在 2006 年 2 月 1 日重新为甲签发了一个序列号为 12346 的数字证书，有效期到 2020 年 1 月 1 日。自此，甲和 CA 都没有对序列号为 12346 的证书进行过更新或者撤销。由于某些纠纷，在 2007 年 1 月 1 日的时候，乙希望验证文件 A 上的甲的签名。显然乙无法用甲现有的序列号为 12346 的证书验证文件 A 上的签名。但是我们知道，已经作废的序列号为 12345 的证书是可以用来成功验证此签名的。因此，PKI 系统应当对这份作废的证书归档处理，此外，在验证文件 A 上的签名时，签名发生当天的 CRL 也应该从归档中被提出，来证明在 2005 年 1 月 1 日时，序列号为 12345 的证书是合法有效的。可见，如果 PKI 系统没有对旧的证书和 CRL 进行归档处理，那么甲在文件 A 上的签名将无法得

到验证，甲也就可以抵赖这份签名，PKI 系统在这种情况下就失去了它应有的服务功能。

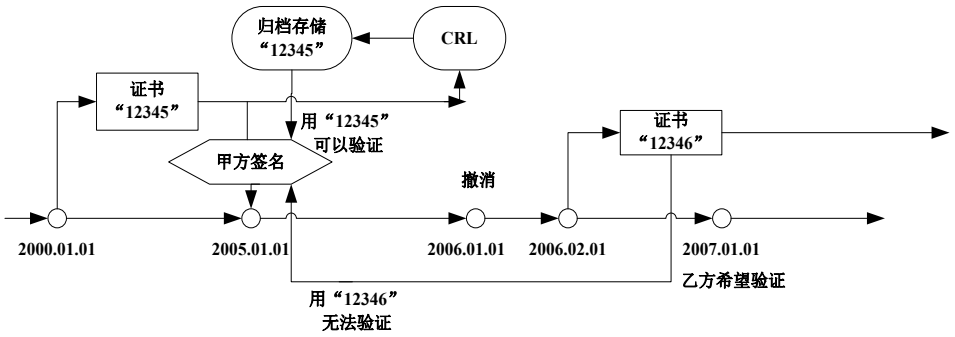


图 14 归档证书的用途

证书的归档没有固定的形式，但是它确实是必须的，PKI 系统必须支持对曾有数据的归档处理，以能在需要的时候为 PKI 系统依赖方找到所需要的旧的数字证书和 CRL 为原则。

3.2.3. 数字证书密码算法

本章节总结了数字证书中的算法，分析其优劣，研究迁移替换思路。

3.2.3.1. 数字证书签名算法

经典签名算法在公钥密码学的发展历程中经历了多个重要阶段，广泛应用于身份验证、数据完整性和非否认性等领域。在 1977 年，RSA 算法由 Ron Rivest、Adi Shamir 和 Leonard Adleman 首次提出。这一算法以大数分解的困难性为基础，成为数字签名的一个重要标准。RSA 在数字证书（如 X.509）中得到广泛应用，然而，由于其密钥长度较长和计算开销大的缺陷，逐渐暴露出在资源受限环境中的局限性[34]。在 1991 年，数字签名算法 DSA 被美国国家标准局（NIST）提出，作为数字签名标准（DSS）的一部分。DSA 设计旨在提高签名生成的速度，其安全性基于离散对数问题。尽管 DSA 在许多应用中得到了广泛采用，但其密钥长度限制及安全性依赖于密钥管理的复杂性，使得该算法在实际使用中面临挑战[30]。2005 年，椭圆曲线数字签名算法 ECDSA 逐渐兴起。ECDSA 利用椭圆曲线理论，允许在提供相同安全级别的情况下使用更短的密钥长度，从而显著提高了签名的效率。随着移动设备和物联网的普及，ECDSA 因其高效性和安全性而成为现代安全协议（如 TLS、SSH 等）的主流签名算法[32]。在 2006 年，中国国家密码管理局发布了商用密码算法（国密），包括 SM2 数字签名算法。SM2 基

于椭圆曲线，专为满足中国市场需求而设计，提供了高效的安全保障。SM2 算法不仅在签名速度和密钥长度上具备优势，还被广泛应用于电子支付和数字证书等领域。在 2010 年，随着计算能力的提升和量子计算的研究进展，传统签名算法的安全性受到威胁。量子计算机的出现可能利用 Shor 算法在多项式时间内破解 RSA 和 ECDSA，使得这些算法面临重大挑战。这一时期，学术界开始关注抗量子算法的研究，以寻求更安全的替代方案。目前，抗量子密码学的研究逐渐成为热点，研究者们开始探索基于格的签名算法和哈希基算法等新兴方案，以增强数字签名的安全性。同时，NIST 等组织启动了后量子密码学的标准化进程，推动了抗量子算法的研究与应用。

在此基础上，我国于 2006 年由国家密码管理局正式发布国产商用椭圆曲线数字签名算法 SM2，形成了适配国内网络安全体系的国密签名标准。SM2 算法在数学结构上与 ECDSA 相似，但采用自主设计的素数椭圆曲线参数与签名校验机制，规避了国际通用曲线可能存在的后门风险，同时优化了签名生成与验证流程，在同等安全强度下具备更优的运算效率。在 X.509 v3 证书体系中，SM2 算法被正式纳入国密证书规范，形成了标准化的 SM2 数字证书体系。不同于传统 RSA、ECDSA 证书统一采用单一密钥完成签名与加密，SM2 证书体系采用签名证书与加密证书双证书分离机制，通过证书扩展字段区分密钥用途，严格隔离身份签名认证与数据加密功能，进一步提升证书体系的安全性与合规性。目前，基于 SM2-SM3 签名套件的国密证书已广泛应用于政务内网、金融支付、国密 TLS 通信及国产化基础设施中，成为我国 PKI 体系的核心签名算法。

3.2.3.2. 数字证书杂凑算法

杂凑函数的发展历程贯穿密码学技术的演进，并在 X.509 证书的签名、验证和完整性保护中发挥了关键作用。从 20 世纪 70 年代起，杂凑函数逐渐被设计并应用，最早的 MD 系列如 MD2 由 Ronald Rivest 于 1979 年提出，主要用于资源受限环境的完整性校验[35]。此后，MD4[36]和 MD5[37]进一步提升了效率，成为 X.509 证书签名的早期主流选择。然而，随着碰撞攻击的研究进展，MD5 逐渐被证明不够安全，无法满足高安全性场景需求。

为弥补 MD 系列算法的缺陷，NIST 在 1993 年发布了 SHA 系列算法[38]。SHA-0 是第一个安全散列标准，但由于发现设计缺陷，同年发布的改进版 SHA-

1 迅速取代了其位置。SHA-1 凭借其良好的性能和抗碰撞性，成为 90 年代和 21 世纪初主流的杂凑函数，并广泛用于 X.509 证书的签名生成。然而，2005 年，Wang Xiaoyun 团队首次提出了有效的 SHA-1 碰撞攻击方法，显示其安全性不再足够。2017 年，Google 进一步实现了完整的 SHA-1 碰撞攻击，这标志着 SHA-1 被完全废弃，推动行业转向更安全的算法。

为了应对安全需求的提升，NIST 在 2001 年推出了 SHA-2 系列算法[38]，包括 SHA-224、SHA-256、SHA-384 和 SHA-512。SHA-2 采用更复杂的设计，大幅提升了抗碰撞性，其中特别是 SHA-256，已成为当前 X.509 证书的标准杂凑算法，被广泛应用于数字签名、证书链验证等场景中。然而，随着量子计算的发展，传统杂凑算法的安全性可能受到威胁，这促使研究人员进一步寻求更先进的解决方案。

2012 年，NIST 通过竞赛推出了 SHA-3 作为新的安全散列标准。SHA-3 基于海绵结构设计，与 SHA-2 相比具备更高的灵活性和抗攻击能力。虽然 SHA-3 已经成为标准，但由于 SHA-2 目前的安全性足够高，且已在 X.509 证书中广泛部署，因此 SHA-3 的实际应用尚未全面普及。与此同时，轻量级杂凑算法的需求日益增长，如 BLAKE2 和 BLAKE3，这些算法在保持高效性能的同时提供了良好的安全性，也逐渐被考虑用于物联网设备和证书应用。

此外，抗量子密码学的兴起也引发了对新型杂凑函数的研究，例如基于哈希的签名方案（如 SPHINCS+）已经成为抗量子安全领域的研究热点。这些算法不仅增强了抗量子计算威胁的能力，还提供了未来高安全性数字签名的可能性。

总体来看，杂凑函数的研究和演进不断适应新的安全挑战，从 MD 系列到 SHA 系列，再到现代的抗量子和轻量级设计，其在 X.509 证书中的应用为数字身份认证的完整性和可信性提供了坚实的技术支撑。

4. 方案设计

为应对量子计算对传统 RSA、经典椭圆曲线密码体系带来的破解威胁，同时兼顾现有 PKI 基础设施、TLS 通信网络的向下兼容性，本文围绕适配 TLS 协议的混合抗量子证书体系完成整体方案设计，从身份签名安全加固与证书结构

扩展改造两大核心维度开展落地实现。方案摒弃纯后量子密码直接替换的激进升级思路，采用经典密码算法与 NIST 标准化后量子密码算法融合的渐进式改造思路，在完整保留 X.509v3 证书原有规范、标准证书签发校验流程的基础上，分别完成混合双重签名机制定制设计与扩展型混合证书格式设计。一方面通过双算法联合签名提升证书链路的量子安全可信能力，保证整条证书信任链从根证书到终端实体证书均可抵御 Shor 算法暴力破解；另一方面依托自定义 ASN.1 对象标识符扩展字段承载后量子公钥与签名数据，不改动原有证书基础结构与解析逻辑，实现与存量网络设备、传统校验组件的无缝兼容。后续将分别从混合签名机制设计、混合证书扩展格式设计两个层面，详细阐述方案的具体结构、实现逻辑与代码落地思路。

4.1. 混合签名设计

结合抗量子密码迁移中出现的问题，本课题在方案设计过程中重点关注证书生成工具的安全性、抗量子签名算法的性能以及尺寸优化问题。通过对数字证书签发速度和证书大小的需求分析，进一步明确了对密码算法参数的要求，并据此选择适配的密码算法及参数组合。在此基础上，结合现有产品的兼容性和未来业务扩展需求，研究设计了适用于抗量子/混合证书格式的技术方案；同时开发支持多种证书格式的证书签发工具，确保其能够支持至少一种抗量子密码算法及其多种参数配置，以满足密码协议试验验证的需求。

针对抗量子密码算法中存在的公钥-密文尺寸不平衡、功能与现有应用场景不匹配等问题，本课题重点研究了银行业务场景下的抗量子证书结构特性及性能特征。依据 GB/T 20518-2018 和 RFC 5280 标准，构建符合具体应用场景计算性能要求的抗量子密码协议安全属性规范集合。这一工作为后续的抗量子证书设计提供了理论基础和技术指导。

在抗量子认证授权模块的设计过程中，密码算法的性能直接决定了整体方案的效果。本课题结合现有认证授权模块的安全性和性能调研结果，在保证安全性和性能不低于现有水平的前提下，通过代码优化、算法改进以及硬件加速等技术手段，进一步提升了模块的吞吐量和计算效率。针对经典计算与抗量子计算的融合需求，课题组提出了两种混合签名方案：

可分离混合签名：该方案基于经典密码算法和抗量子密码算法的独立性设计。具体而言，采用基于经典密码算法的签名与基于抗量子密码算法的签名直接拼接的方式生成混合签名。其显著特点是抗量子签名与经典签名可以并行计算，且不对原有的经典签名验签流程产生影响，仅在原有功能基础上叠加了抗量子签名验签能力。

不可分离混合签名：该方案采用嵌套式设计，首先使用经典密码算法生成签名，随后利用抗量子密码算法对经典签名进行再次签名以生成最终的抗量子混合签名。这种设计方式的优势在于，抗量子算法能够有效保护基于经典算法的签名的完整性和真实性，从而显著提升了整体安全性。然而，该方案在一定程度上牺牲了签名效率。

如图 15 所示，可分离混合签名方案分别独立生成经典签名和抗量子签名，显著提升了系统的性能和响应速度。与不可分离模式相比，这种并行计算的方式能够有效减少延迟，提高处理效率，尤其适用于金融交易、实时数据处理等高并发场景中的快速响应需求。

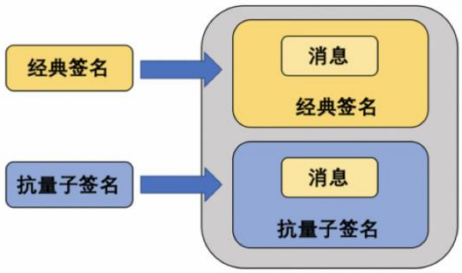


图 15 可分离混合签名

如图 16 所示，不可分离混合签名方案采用嵌套式设计，首先利用经典密码算法生成初始签名，随后基于抗量子密码算法对该经典签名进行再次签名，从而形成最终的抗量子混合签名。这种设计方式通过将经典签名与抗量子签名紧密耦合，显著增强了整体安全性。具体而言，抗量子算法能够有效保护基于经典算法的签名的真实性和完整性，防止签名被篡改或伪造。然而，由于嵌套式签名的过程需要顺序执行，该方案在一定程度上牺牲了签名效率。

不可分离混合签名的验证流程需要顺序进行经典签名再进行抗量子签名。这种设计虽然降低了系统的灵活性，但在安全性方面具有显著优势。特别是在金融交易、高价值数据传输等需要高度安全防护的场景中，该方案能够有效防止签名被篡改或伪造，确保数据的完整性和真实性。因此，不可分离混合签名方案特别

适用于对安全性要求极高的业务场景，其设计思路为未来抗量子密码迁移提供了重要的技术参考和实践依据。同时，该方案还具有一定的扩展性，能够在后量子时代支持更复杂的签名验证需求。

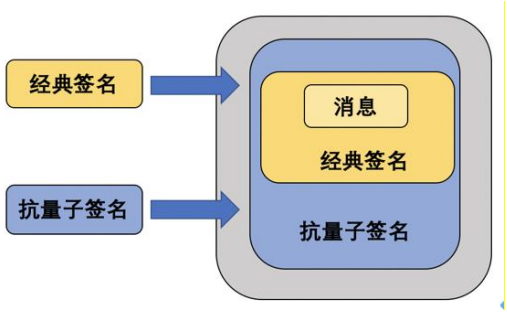


图 16 不可分离混合签名

4.2. 混合证书格式设计

本课题基于混合使用拓展的证书格式设计，提出了两种不同的混合证书格式：可分离混合证书和不可分离混合证书。这两种证书格式分别针对不同的应用场景和技术需求，通过优化证书结构和签名机制，实现了对经典密码算法与抗量子密码算法的支持。

如图 17 所示，可分离混合证书的设计思路是采用可分离混合签名方案，将经典签名值 $sig_1(m)$和抗量子签名值 $sig_2(m)$分别存储于不同的域中。具体而言，经典签名值被放置在证书的签名域中，而经典公钥则位于基本信息域的公钥字段内。与此同时，抗量子签名值和抗量子公钥被独立地存放于拓展域中。这种设计方式充分利用了证书结构的灵活性，在确保兼容现有证书解析机制的同时，为抗量子密码算法的集成提供了扩展空间。混合证书格式的设计具有以下显著特点：首先，其能够实现经典签名和抗量子签名的独立验证，无需对原有证书解析流程进行大幅调整，从而降低了迁移成本。其次，在安全性方面，该设计通过将抗量子签名与经典签名分开存储，既保留了经典密码算法的兼容性，又增强了整体的安全防护能力。最后，这种结构还为未来从经典密码向抗量子密码的完全过渡提供了灵活性和可扩展性，确保证书格式能够在后量子时代继续满足实际应用需求。

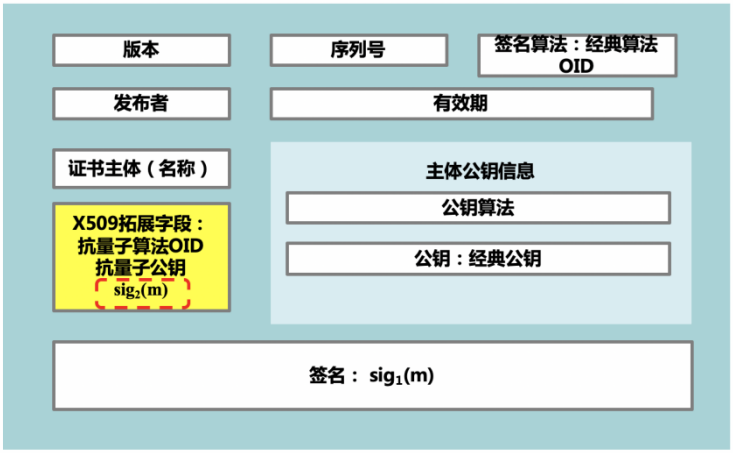


图 17 混合证书结构

4.3. 公钥证书链设计

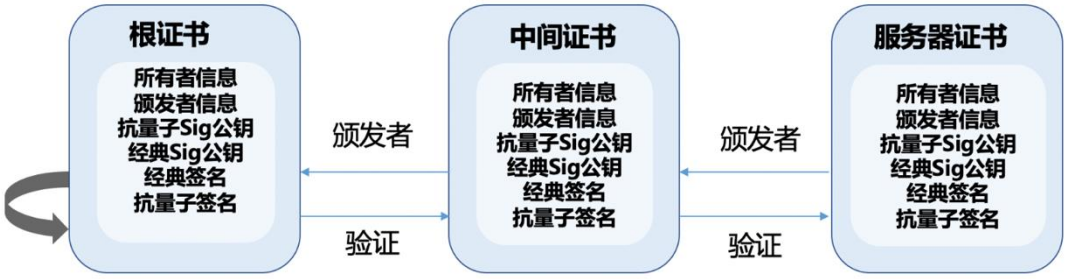


图 18 混合证书链结构设计

本文所设计的混合抗量子证书链采用根证书、中间证书、服务器终端证书三级分层架构，整体遵循扩展后的 X.509v3 证书规范完成改造，完整保留传统 PKI 体系层级信任传递逻辑，同时在每一级证书内部融合经典签名体系与后量子抗量子签名体系，实现整条信任链路全节点量子安全防护。整条证书链以自签名根证书作为全局信任锚点，根证书仅对下级中间证书完成签发授权，中间证书承接根证书的信任背书，进一步为业务场景下的服务器终端证书颁发身份凭证，终端服务器证书则直接用于 TLS 握手过程中的实体身份认证，三级证书自上而下形成完整的签发授权链路，自下而上完成逐级签名校验，构建起兼容现有网络环境的量子安全信任体系。

整条证书链内的每一级证书均同步搭载两套独立签名密码体系，单张证书内同时存储经典 Sig 公钥与抗量子 Sig 公钥两类公钥载体，在证书签发环节，上级签发实体分别使用自身经典私钥、抗量子私钥对下级证书完整内容生成双重签名，并将两组签名结果一并写入下级证书字段。在证书合法性校验阶段，校验方会依次提取证书内经典公钥、抗量子公钥，分别对两组签名数据独立核验，只有

双重签名全部校验通过，才会认定该证书链路可信，这种双公钥、双签名的混合设计既能够依托 RSA、ECDSA 等传统密码算法适配存量不支持后量子密码的终端设备，保障与现有网络基础设施的向下兼容性，又依靠 ML-DSA 标准化抗量子签名算法抵御 Shor 量子算法带来的破解风险，实现传统安全能力与量子安全防护能力的同步覆盖。

作为整条信任体系的信任锚，根证书采用自签名设计，依靠自身内置的经典公钥与抗量子公钥完成自校验，无需依赖上级证书即可确立信任根基，根证书内置的双密码公钥与双重签名规则会完整向下传递至中间证书与服务器证书，保证从信任源头到业务终端的全链路无断点量子安全防护。中间证书作为层级过渡节点，承担信任分发与业务隔离的作用，能够通过分层架构避免根证书私钥频繁参与在线签发操作，降低最高层级信任锚的密钥泄露风险；面向通信业务场景的服务器终端证书直接对接 TLS 协议握手流程，在 Certificate 握手消息中完整上传整套混合证书链，客户端在身份认证阶段逐级完成根证书、中间证书、服务器证书的双重签名校验，在不重构原有 TLS 证书校验流程、不改动 PKI 基础架构的前提下，完成具备长期量子安全能力的身份鉴别，为混合抗量子 TLS 协议的渐进式落地部署提供完整可信证书支撑。

4.4. 抗量子证书生成工具设计

在公钥证书格式设计的基础上，本节对 X.509 和 GB/T 25518 国密证书抗量子混合证书生成工具进行设计和实现。工具融合传统密码与后量子密码，通过"传统+抗量子"双栈架构，在保证现有公钥基础设施兼容性的同时，提供量子安全的证书服务。工具支持 X.509v3 证书扩展及双栈密钥生命周期管理。

抗量子/混合公钥证书生成工具简化过程大致如下：

1. 用户生成传统公钥密码签名密钥对、抗量子密码签名密钥对和抗量子密码加密临时密钥对等信息，然后生成混合签名证书请求文件，包含用户的身份信息、传统公钥密码签名密钥对、抗量子密码签名密钥对和抗量子密码加密临时密钥对等信息，并将请求文件发送给证书颁发机构。

2. 证书颁发机构负责接收用户的请求文件，并检查用户的身份证明，包括用户身份信息核实、利用抗量子密码签名公钥和传统公钥密码签名公钥分别验证

抗量子密码签名和传统密钥签名的有效性。验证通过后，向密钥生成中心请求生成用户的密钥对。

3. 密钥生成中心根据证书颁发机构的请求，运行传统公钥密码密钥生成算法和抗量子密码加密密钥生成算法，为用户生成相应的密钥对，并返回给证书颁发机构。

4. 证书生成模块负责根据各种密钥、用户身份信息等生成混合加密私钥、混合加密公钥、私钥加密密文和混合密文，根据相关信息生成混合签名自签证书、用户签名证书和用户加密证书，并将相关信息发送给用户端。

5. 用户端在收到证书和密钥等信息后，对签名证书和加密证书进行验证，以确保证书的合法性和安全性。验证通过后，用户可以使用抗量子混合证书进行相关的安全通信或其他操作。如果验证不通过，则返回错误信息，提示用户重新进行相关操作。

在上述过程中，主要包括了 3 种执行不同功能的实体：

证书颁发机构：具有自己的公私密钥对，负责为其他人签发证书，用自己的密钥来证实用户的公钥信息。

用户：拥有自己的证书和与证书中公钥匹配的私钥，被称为证书持有者。证书持有者的身份信息和对应的公钥会出现在证书中。

密钥生成中心：负责生成和管理加密公钥和私钥对。这包括密钥的创建、存储、分发和撤销。与证书颁发机构紧密合作，完成数字证书签发。

证书颁发机构、密钥生成中心和用户共同组成了抗量子/混合公钥证书生成工具。即，抗量子/混合公钥证书生成工具中的最基本功能组件（简称为基本组件）有三个，分别为证书颁发机构、密钥生成中心和用户。

抗量子/混合公钥证书生成工具架构图如下所示：

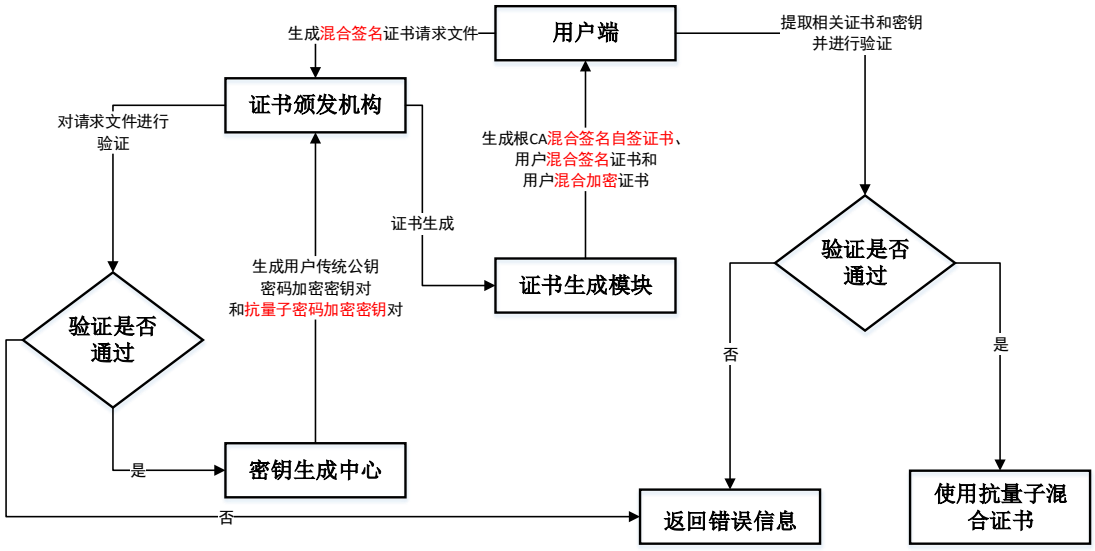


图 19 抗量子/混合公钥证书生成工具架构图

抗量子/混合公钥证书生成工具基本流程为：

1. 用户端生成混合证书请求文件，包含身份信息、传统与抗量子密钥对及双重签名。
2. 用户端发送请求文件至 CA。
3. CA 验证请求文件。
4. 使用传统算法验证传统签名。
5. 使用抗量子算法验证抗量子签名。
6. 验证通过后，CA 调用 KGC 生成用户密钥对。
7. KGC 返回密钥对至 CA。
8. CA 生成混合证书链，包含用户证书、中间证书及根证书。
9. CA 将证书链及加密私钥密文发送至用户端。
10. 用户端使用验证模块验证证书链及私钥密文。
11. 验证通过后，用户端使用私钥进行解密和签名操作。

5. 抗量子证书生成工具模块设计

5.1. 系统整体分层架构

本系统基于 Go 语言开发，依托 Cloudflare Circl 密码库实现传统密码与后量子密码融合的混合 X.509v3 证书链生成能力。为实现代码解耦、功能易扩展、

逻辑分层清晰的开发目标，系统采用自上而下四层分层架构进行模块化设计如图 20 所示，整体架构分为入口调度层、证书生成核心层、密码算法支撑层、公共工具支撑层，各层级单向依赖，上层业务模块仅调用下层基础能力，底层模块不感知上层业务逻辑。

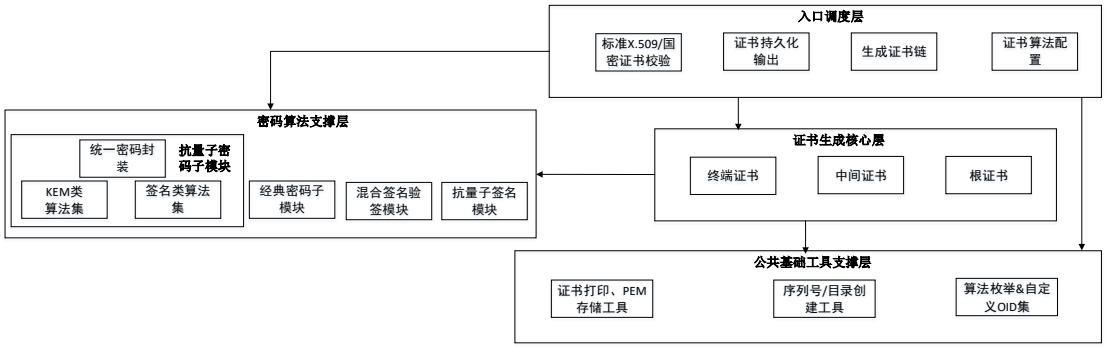


图 20 抗量子证书生成工具整体架构图

从架构图可以清晰看出各层级的包含关系与调用依赖关系。最顶层为入口调度层，作为整个程序的统一执行入口，统筹全部证书生成流程；第二层为证书生成核心层，承载证书链签发的核心业务逻辑，是系统业务主体；第三层密码算法支撑层为上层提供传统密码、后量子密码、混合签名底层密码能力；最底层公共工具支撑层提供全局通用工具函数，被上层所有模块共享调用。证书生成核心层执行签发逻辑时，会依赖密码算法支撑层生成密钥与完成后量子签名，同时所有层级在运行过程中均会调用公共工具层的通用辅助函数，完成路径创建、序列号生成、证书存储打印等操作。整套分层架构将业务逻辑、密码底层、通用工具完全隔离，新增后量子算法、扩展证书模板、新增证书管理功能时仅需修改对应层级模块，不会对其他层级代码造成侵入性改动，系统可维护性与扩展性显著提升。

5.2. 各层级模块详细功能设计

5.2.1. 公共工具支撑层

公共工具支撑层是系统最底层基础组件，不依赖任何其他业务模块，为全系统提供可复用的通用辅助能力，主要包含三类功能单元。第一类为常量与枚举定义单元，定义 PQAlgorithm 枚举统一管理系统支持的全部后量子密码算法，同时自定义两组 ASN.1 标准 OID 标识符，分别用于在 X.509v3 扩展字段存储后

量子公钥原始字节、后量子签名结果，是实现混合证书扩展字段的基础标识；第二类为通用数据生成与环境工具，实现 128 位随机证书序列号生成函数，保证每张证书序列号全局唯一，同时提供目录自动创建函数，程序启动时自动检测并生成证书输出文件夹，避免文件写入报错；第三类为证书 IO 工具单元，封装证书控制台打印函数，可输出证书主体名称、签发方、有效期、CA 标识等关键信息，同时实现标准 PEM 格式证书持久化存储函数，将 DER 编码证书写入本地文件，统一管理证书输出路径。

该层所有工具函数无业务耦合，仅提供独立通用能力，上层任意模块均可直接调用，有效消除重复代码，降低整体代码冗余度。

5.2.2. 密码算法支撑层

密码算法支撑层是系统的密码能力底座，为上层证书签发模块提供传统密钥、后量子密钥、混合签名三大核心密码能力，分为传统密码子模块、后量子密码子模块、混合签名工具三部分。

传统密码子模块仅实现 RSA-2048 公私钥生成逻辑，用于生成标准 X.509 证书签发所需传统签名密钥，保证生成的混合证书能够兼容现有标准 PKI 校验流程，存量业务系统可通过原生 RSA 公钥完成基础证书合法性校验。

后量子密码子模块基于 Circl 密码库封装统一结构体 PQKeyPair，兼容签名类、KEM 密钥交换类两类后量子算法，统一存储算法标识、公私钥实例、公钥序列化字节数组。模块分别实现 CA 签发专用 Dilithium、SPHINCS 签名密钥生成逻辑，以及终端实体专用 Kyber 密钥交换密钥生成逻辑，支持 Dilithium2、Dilithium3、SPHINCS128s、SPHINCS128f、Kyber512、Kyber768 六类标准化后量子算法，屏蔽不同后量子算法底层 API 差异，向上层提供统一的密钥生成接口。

混合签名模块提供通用签名函数 pqSign，接收上层传入的后量子密钥与证书 TBS 待签数据，自动识别密钥绑定的算法类型并调用对应底层签名逻辑，输出后量子签名字节流，统一上层证书签发模块的签名调用方式，简化证书签发流程的代码实现。

5.2.3. 证书生成层

证书生成核心层为系统业务核心，包含根证书生成、中间 CA 证书生成、终端实体证书生成三个独立子模块，自上而下完成根 CA、两级中间 CA、终端实体的四级混合抗量子证书链签发，全部证书遵循扩展 X.509v3 规范完成封装。

根证书生成模块用于生成全局信任锚自签名证书，同步生成 RSA 传统密钥与指定后量子签名密钥，构造证书模板并写入自定义 OID 扩展字段存储后量子公钥；流程上先通过标准 RSA 私钥完成基础证书签发，再调用混合签名模块对证书 TBS 内容生成后量子签名，将签名数据写入新增扩展域，最终输出同时具备传统 PKI 信任体系与抗量子安全能力的根证书。

中间 CA 证书生成模块承接上级 CA 信任链路，复用双层签发逻辑，依托上层 CA 的 RSA 私钥完成标准证书签发，同时使用上层 CA 后量子私钥追加后量子签名；模块配置证书路径长度约束，通过分层 CA 隔离根证书在线签发场景，降低最高信任锚私钥泄露风险，可灵活配置每一级中间 CA 使用的后量子签名算法。

终端实体证书生成模块面向 TLS 通信业务场景设计，关闭 CA 标识，限定密钥用途为数字签名与密钥加密，支持签发纯 Kyber KEM、纯 Dilithium/SPHINCS 签名两类终端混合证书，分别适配密钥协商、身份认证业务场景。所有子模块统一封装扩展字段写入逻辑，每张证书均携带自定义扩展存储后量子公钥与上级后量子签名，完整保留 X.509 标准字段，实现与现有网络基础设施的向下兼容。

5.2.4. 接口调度层

入口调度层作为程序统一调度入口，承担证书链参数配置、证书批量生成、持久化存储、证书链校验四大职能。程序启动后首先调用公共工具层接口初始化证书存储目录，预先配置根 CA、一级中间 CA、二级中间 CA 分别使用的后量子算法，依次串行调用根证书、两级中间 CA 生成接口，完成整条 CA 信任链路构建；链路搭建完成后，循环遍历全部支持的后量子算法，批量生成多类型终端实体混合证书，每生成一张证书自动调用打印、存储工具完成信息输出与本地持久化。

全部证书生成完毕后，主程序构建标准 X.509 证书信任池，将根证书存入根信任池、两级中间证书存入中间证书池，调用终端证书原生校验接口完成整条

证书链合法性验证，验证混合扩展证书可被传统 PKI 校验逻辑正常解析，最终输出程序执行完成提示。该层集中管控整套证书链生成流程，仅需修改顶层算法配置参数，即可快速调整证书链各级节点密码组合，适配不同安全等级、不同业务场景的混合证书生成需求。

5.3. 模块整体执行流程

程序启动后由入口调度层初始化运行环境，按照预设算法参数触发根证书生成流程；根证书模块调用密码算法支撑层生成 RSA 密钥与 Dilithium2 后量子密钥，构造证书模板写入后量子公钥扩展，经 RSA 签发、后量子二次签名后完成根证书输出。随后主程序传入根证书与密钥信息调用一级中间 CA 生成模块，复用双层签名签发逻辑生成 SPHINCS 算法中间证书，再基于一级中间节点生成 Dilithium3 二级中间证书，完成完整 CA 信任链路搭建。链路构建完成后，主程序循环调用终端证书生成模块，批量生成 Kyber、Dilithium、SPHINCS 多类型终端混合证书，每张证书自动保存至本地证书目录。全部证书生成结束后，主程序加载证书构建信任池，执行标准 X.509 证书链校验，验证混合证书与传统 PKI 体系的兼容性，整套流程闭环完成混合抗量子四级证书链生成与功能验证。

6. 抗量子证书生成工具实现

结合前文提出的基于 X.509 v3 扩展字段增量嵌入抗量子密码数据的轻量化改造思路，设计并实现一套兼容标准 PKI 体系的抗量子证书生成工具。

兼容性优先：完整保留原始 X.509 证书基础字段、RSA 主体公私钥体系，仅通过自定义私有扩展字段存储后量子公钥与后量子签名，不改动证书基础解析逻辑，兼容传统 TLS、PKI 验证链路；

多算法适配：支持 NIST 标准化后量子算法，包括格基签名算法 Dilithium2/3、哈希基签名算法 SPHINCS+（SLH-DSA）、格基密钥封装机制 Kyber512/768；

层级证书链构建：支持根 CA、两级中间 CA、终端实体证书的完整层级签发，依托父级节点后量子私钥完成扩展字段内抗量子签名，构建混合信任链；

可验证性：生成证书可通过标准 x509 库完成基础证书链校验，后量子密码材料独立存储于扩展项，实现传统密码与后量子密码双信任体系并行。

6.1. 整体架构

工具基于 Go 语言开发，依托 cloudflare/circl 后量子密码库实现 NIST 标准化 PQC 算法调用，基于标准 crypto/x509 库完成 X.509 证书模板构建、DER 编码与 PEM 持久化。整体架构分为五层：

算法抽象层：定义后量子算法枚举 PQAlgorithm，封装 Dilithium、SPHINCS、Kyber 三类算法的密钥生成、序列化、签名接口；

密钥管理层：封装 PQKeyPair 结构体，统一存储签名公私钥、KEM 封装公私钥、公钥二进制序列化数据；

证书模板层：基于 X.509 v3 版本构造证书模板，通过自定义 OID 注册两个私有扩展项，分别用于存放抗量子公钥、抗量子签名值；

证书签发层：分层实现根 CA、中间 CA、终端实体证书签发逻辑，先完成标准 RSA 证书基础签发，再追加后量子扩展字段与 PQC 签名；

验证与持久层：实现证书本地目录存储、基础证书链合法性校验、证书信息格式化打印功能。

6.2. 数据结构和基础算法

枚举所有支持的 NIST 后量子候选算法，区分签名类算法、密钥封装类算法，用于密钥生成与分支逻辑调度：

```
type PQAlgorithm int

const (
    Dilithium2 PQAlgorithm = iota
    Dilithium3
    SPHINCS128s
    SPHINCS128f
    Kyber512
    Kyber768
)
```

其中：Dilithium、SPHINCS 为签名算法，用于 CA 节点身份签名背书；Kyber 为密钥封装算法，用于终端实体密钥协商场景。

PQKeyPair 作为通用容器，兼容签名算法、KEM 算法两类密钥结构，统一管理密钥实例与序列化公钥字节，屏蔽不同 PQC 算法的底层结构差异：

```
type PQKeyPair struct {
    Algo        PQAlgorithm     // 绑定算法类型
    SignSK      interface{}      // 签名私钥（Dilithium/SPHINCS）
    SignPK      interface{}      // 签名公钥
    Scheme      sign.Scheme      // SPHINCS 算法抽象方案句柄
    KemSK       interface{}      // Kyber 解密私钥
    KemPK       interface{}      // Kyber 封装公钥
    PubBytes []byte             // 公钥统一二进制序列化结果
}
```

6.3. 抗量子密码生成模块

CA 证书仅使用签名类后量子算法，提供层级签名背书能力，分为 Dilithium 原生密钥生成、SPHINCS 通过方案名动态加载两种实现：

```
func generateCAKey(algo PQAlgorithm) *PQKeyPair {
    switch algo {
    case Dilithium2:
        pk, sk, _ := mode2.GenerateKey(rand.Reader)
        return &PQKeyPair{Algo: Dilithium2, SignSK: sk, SignPK: pk, PubBytes: pk.Bytes()}
    case Dilithium3:
        pk, sk, _ := mode3.GenerateKey(rand.Reader)
        return &PQKeyPair{Algo: Dilithium3, SignSK: sk, SignPK: pk, PubBytes: pk.Bytes()}
    case SPHINCS128s, SPHINCS128f:
        scheme := getSPHINCSScheme(algo)
        pk, sk, err := scheme.GenerateKey()
```

```
            if err != nil {
                panic(err)
            }
            pubBytes, err := pk.MarshalBinary()
            if err != nil {
                panic(err)
            }
            return &PQKeyPair{
                Algo:        algo,
                Scheme:     scheme,
                SignPK:     pk,
                SignSK:     sk,
                PubBytes: pubBytes,
            }
        default:
            panic("不支持的 CA 算法")
        }
    }
```

终端实体同时支持 KEM 密钥协商算法 Kyber 与签名算法，区分密钥封装公私钥、签名公私钥两类存储逻辑：

```
    func generateEndEntityKey(algo PQAlgorithm) *PQKeyPair {
        switch algo {
        case Kyber512:
            pk, sk, _ := kyber512.GenerateKeyPair(rand.Reader)
            pubBytes, _ := pk.MarshalBinary()
            return &PQKeyPair{Algo: Kyber512, KemPK: pk, KemSK: sk, PubBytes: pubBytes}
        case Kyber768:
            pk, sk, _ := kyber768.GenerateKeyPair(rand.Reader)
```

```
            pubBytes, _ := pk.MarshalBinary()
            return &PQKeyPair{Algo: Kyber768, KemPK: pk, KemSK: sk, PubBytes:
pubBytes}
        case Dilithium2:
            pk, sk, _ := mode2.GenerateKey(rand.Reader)
            return &PQKeyPair{Algo: Dilithium2, SignPK: pk, SignSK: sk,
PubBytes: pk.Bytes()}
        case Dilithium3:
            pk, sk, _ := mode3.GenerateKey(rand.Reader)
            return &PQKeyPair{Algo: Dilithium3, SignPK: pk, SignSK: sk,
PubBytes: pk.Bytes()}
        case SPHINCS128s, SPHINCS128f:
            scheme := getSPHINCSScheme(algo)
            pk, sk, err := scheme.GenerateKey()
            if err != nil {
                panic(err)
            }
            pubBytes, err := pk.MarshalBinary()
            if err != nil {
                panic(err)
            }
            return &PQKeyPair{
                Algo:      algo,
                Scheme:    scheme,
                SignPK:    pk,
                SignSK:    sk,
                PubBytes: pubBytes,
            }
        default:
```

```
            panic("不支持的终端算法")
        }
    }
```

6.4. 分层证书签发实现

根 CA 为信任链起点，自签发标准 RSA 证书，首次签发时将抗量子公钥写入扩展字段；二次更新模板时，使用根节点后量子私钥对证书主体信息（RawTBSCertificate）签名，追加抗量子签名扩展项，完成双体系自信任：

```
func genRootCA(algo PQAlgorithm) (*rsa.PrivateKey, *x509.Certificate,
*PQKeyPair) {
    rsaSK := genRSA()
    pq := generateCAKey(algo)
    // 构造 X.509 v3 根证书模板
    tpl := &x509.Certificate{
        Version:                 3,
        SerialNumber:            generateSerial(),
        Subject:                 pkix.Name{CommonName: "PQ-Root-CA"},
        Issuer:                  pkix.Name{CommonName: "PQ-Root-CA"},
        NotBefore:               time.Now(),
        NotAfter:                time.Now().Add(20 * 365 * 24 * time.Hour),
        IsCA:                    true,
        KeyUsage:                x509.KeyUsageCertSign |
x509.KeyUsageCRLSign,
        BasicConstraintsValid: true,
        MaxPathLen:              3,
        // 首次写入：抗量子公钥扩展
        ExtraExtensions: []pkix.Extension{{
            Id:      OIDQuantumPublicKey,
            Value: pq.PubBytes,
        }},
```

```
    }
    // 第一轮：标准 RSA 自签基础证书
    der, err := x509.CreateCertificate(rand.Reader, tpl, tpl, &rsaSK.PublicKey,
rsaSK)
    if err != nil { panic(err) }
    c, err := x509.ParseCertificate(der)
    if err != nil { panic(err) }

    // 使用根 CA 后量子私钥生成签名，追加抗量子签名扩展
    sig := pqSign(pq, c.RawTBSCertificate)
    tpl.ExtraExtensions = append(tpl.ExtraExtensions, pkix.Extension{
        Id:    OIDQuantumSignature,
        Value: sig,
    })
    // 第二轮：更新包含 PQC 签名的最终证书
    der, err = x509.CreateCertificate(rand.Reader, tpl, tpl, &rsaSK.PublicKey, rsaSK)
    if err != nil { panic(err) }
    c, err = x509.ParseCertificate(der)
    if err != nil { panic(err) }
    return rsaSK, c, pq
}
```

中间 CA 由根 CA 背书、终端证书由二级中间 CA 背书，签发流程与根 CA 一致：先由父级 RSA 私钥完成标准证书签发，再使用父级后量子私钥生成 PQC 签名并写入扩展项，构建 RSA 传统信任链 + 后量子算法信任链并行的混合证书链。终端证书禁止 CA 属性，密钥用途限定为签名与密钥加密：

```
func genEndEntity(
    parentRsa *rsa.PrivateKey,
    parentCert *x509.Certificate,
    parentPQ *PQKeyPair,
```

```
    algo PQAlgorithm,
    name string,
) *x509.Certificate {
    rsaSK := genRSA()
    pq := generateEndEntityKey(algo)
    tpl := &x509.Certificate{
        Version:                 3,
        SerialNumber:            generateSerial(),
        Subject:                 pkix.Name{CommonName: name},
        Issuer:                  parentCert.Subject,
        NotBefore:               time.Now(),
        NotAfter:                time.Now().Add(2 * 365 * 24 * time.Hour),
        IsCA:                    false, // 终端节点禁止 CA 签发权限
        KeyUsage:                x509.KeyUsageDigitalSignature |
x509.KeyUsageKeyEncipherment,
        BasicConstraintsValid: true,
        ExtraExtensions: []pkix.Extension{{
            Id:    OIDQuantumPublicKey,
            Value: pq.PubBytes,
        }},
    }
    der, err := x509.CreateCertificate(rand.Reader, tpl, parentCert,
&rsaSK.PublicKey, parentRsa)
    if err != nil {
        panic(err)
    }
    c, err := x509.ParseCertificate(der)
    if err != nil {
        panic(err)
```

```
        }
        sig := pqSign(parentPQ, c.RawTBSCertificate)
        tpl.ExtraExtensions = append(tpl.ExtraExtensions, pkix.Extension{
            Id:    OIDQuantumSignature,
            Value: sig,
        })
        der, err = x509.CreateCertificate(rand.Reader, tpl, parentCert,
&rsaSK.PublicKey, parentRsa)
        if err != nil {
            panic(err)
        }
        c, err = x509.ParseCertificate(der)
        if err != nil {
            panic(err)
        }
        return c
    }
```

6.5. 主程序调度与证书链验证

主程序构建三级信任链：根 CA（Dilithium2）→一级中间 CA（Dilithium2）→二级中间 CA（Dilithium2），向下签发 Kyber、Dilithium、SPHINCS 全品类终端证书，覆盖所有支持的后量子算法：

```
    func main() {
        fmt.Println("=== Dilithium + SPHINCS + Kyber 抗量子证书链生成器
===")
        ensureCertDir()
        // 分层算法绑定
        rootAlgo := Dilithium2
        inter1Algo := SPHINCS128s
```

```
    inter2Algo := Dilithium3
    // 生成根、两级中间 CA
    rootRSA, rootCert, rootPQ := genRootCA(rootAlgo)
    i1RSA, i1Cert, i1PQ := genIntermediate(rootRSA, rootCert, rootPQ, "PQ-Intermediate-1-SPHINCS", inter1Algo)
    i2RSA, i2Cert, i2PQ := genIntermediate(i1RSA, i1Cert, i1PQ, "PQ-Intermediate-2-Dilithium", inter2Algo)
    // 批量生成全算法终端证书
    user1 := genEndEntity(i2RSA, i2Cert, i2PQ, Kyber512, "user-kyber512")
    user2 := genEndEntity(i2RSA, i2Cert, i2PQ, Kyber768, "user-kyber768")
    user3 := genEndEntity(i2RSA, i2Cert, i2PQ, Dilithium2, "user-dilithium2")
    user4 := genEndEntity(i2RSA, i2Cert, i2PQ, Dilithium3, "user-dilithium3")
    user5 := genEndEntity(i2RSA, i2Cert, i2PQ, SPHINCS128s, "user-sphincs128s")
    user6 := genEndEntity(i2RSA, i2Cert, i2PQ, SPHINCS128f, "user-sphincs128f")
}
```

6.6. 小结

本章基于 Go 语言与 Cloudflare CIRCL 后量子密码库，完成了一套基于 X.509 v3 扩展字段的抗量子证书生成工具，实现 Dilithium、SPHINCS、Kyber 三类 NIST 标准化后量子算法的密钥管理与证书嵌入。工具严格遵循不修改原有证书基础字段、增量扩展的核心设计原则，将抗量子公钥与签名隔离存储在自定义扩展项中，在保证传统 PKI、TLS 协议完全兼容的前提下，为 X.509 证书提供抗量子安全能力。标准证书链验证结果表明，该改造方案兼容性达标，可作为后量子 TLS 异构认证、新旧密码体系平滑迁移的原型支撑平台，为前文提出的异构认证互操作性鸿沟提供工程实现基础。

7. 参考文献

[1] Shor P W. Algorithms for quantum computation: discrete logarithms and factoring[C]//Proceedings 35th annual symposium on foundations of computer science. Ieee, 1994: 124-134.

[2] Housley R, Polk W, Ford W, et al. RFC3280: Internet X. 509 public key infrastructure certificate and certificate revocation list (CRL) profile[J]. 2002.

[3] Rivest R L, Shamir A, Adleman L. A method for obtaining digital signatures and public-key cryptosystems[J]. Communications of the ACM, 1978, 21(2): 120-126.

[4] Goldreich O, Goldwasser S, Halevi S. Public-key cryptosystems from lattice reduction problems[C]//Advances in Cryptology—CRYPTO'97: 17th Annual International Cryptology Conference Santa Barbara, California, USA August 17–21, 1997 Proceedings 17. Springer Berlin Heidelberg, 1997: 112-131.

[5] Gentry C, Peikert C, Vaikuntanathan V. Trapdoors for hard lattices and new cryptographic constructions[C]//Proceedings of the fortieth annual ACM symposium on Theory of computing. 2008: 197-206.

[6] Lyubashevsky V. Lattice signatures without trapdoors[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012: 738-755.

[7] Ducas L, Durmus A, Lepoint T, Lyubashevsky V. Lattice signatures and bimodal Gaussians[C]//2013 Annual International Cryptology Conference. Springer, 2013: 40-56.

[8] Merkle R C. Secrecy, authentication, and public key systems[M]. Stanford university, 1979.

[9] Lamport L. Constructing digital signatures from a one way function[J]. 1979.

[10] Winternitz R S. A secure one-way hash function built from DES[C]//1984 IEEE Symposium on Security and Privacy. IEEE, 1984: 88-88.

[11] McGrew D, Curcio M, Fluhrer S. RFC 8554: Leighton-Micali hash-based signatures[J]. 2019.

[12] Hülsing A, Butin D, Gazdag S, et al. XMSS: eXtended Merkle signature scheme[R]. 2018.

[13] Matsumoto T, Imai H. Public quadratic polynomial-tuples for efficient signature-verification and message-encryption[C]//Advances in Cryptology—EUROCRYPT'88: Workshop on the Theory and Application of Cryptographic Techniques Davos, Switzerland, May 25–27, 1988 Proceedings 7. Springer Berlin Heidelberg, 1988: 419-453.

[14] Patarin J. Cryptanalysis of the Matsumoto and Imai public key scheme of Eurocrypt'88[C]//Advances in Cryptology—CRYPT0'95: 15th Annual International Cryptology Conference Santa Barbara, California, USA, August 27–31, 1995 Proceedings 15. Springer Berlin Heidelberg, 1995: 248-261.

[15] Patarin J. Hidden fields equations (HFE) and isomorphisms of polynomials (IP): Two new families of asymmetric algorithms[C]//International conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996: 33-48.

[16] Faugere J C, Joux A. Algebraic cryptanalysis of hidden field equation (HFE) cryptosystems using Gröbner bases[C]//Annual International Cryptology Conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003: 44-60.

[17] Kipnis A, Patarin J, Goubin L. Unbalanced oil and vinegar signature schemes[C]//International Conference on the Theory and Applications of Cryptographic Techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999: 206-222.

[18] Ding J, Schmidt D. Rainbow, a new multivariable polynomial signature scheme[C]//International conference on applied cryptography and network security. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005: 164-175.

[19] Schnorr C P. Efficient signature generation by smart cards[J]. Journal of cryptology, 1991, 4: 161-174.

[20] Casar T, Eng P. Public Comments Regarding FIPS 186-1, Digital Signature Standard (DSS)[J].

[21] Sikeridis D, Kampanakis P, Devetsikiotis M. Post-quantum authentication in TLS 1.3: a performance study[J]. Cryptology ePrint Archive, 2020.

[22] O. Project, “liboqs,” https://github.com/open- quantum- safe/liboqs, 2019, Web page. Accessed 2019-02-08.

[23] D. Stebila and M. Mosca, “Post-Quantum Key Exchange for the Internet and the Open Quantum Safe Project,” Cryptology ePrint Archive, Report 2016/1017, 2016, https://eprint.iacr.org/2016/1017.

[24] D. J. Bernstein, “eBACS: ECRYPT Benchmarking of Cryptographic Systems,” https://bench.cr.yp.to/primitives- sign.html, 2019, Web page. Accessed 2019-02-08.

[25] M.-J. O. Saarinen, “Mobile energy requirements of the upcoming NIST post-quantum cryptography standards,” 2019.

[26] Paul S, Kuzovkova Y, Lahr N, et al. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3[C]//Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. 2022: 727-740.

[27] Bindel N, Hale B. A note on hybrid signature schemes[J]. Cryptology ePrint Archive, 2023.

[28] Liu T, Ramachandran G, Jurdak R. Towards Quantum Resilient IoT: A Backward-Compatible Approach to Secure BLE Key Exchange Against Quantum Threats[C]//Proceedings of the 9th ACM/IEEE Conference on Internet of Things Design and Implementation (IoTDI 2024)(IoTDI 24). Institute of Electrical and Electronics Engineers Inc., 2024.

[29] Ducas L, Kiltz E, Lepoint T, et al. Crystals-dilithium: A lattice-based digital signature scheme[J]. IACR transactions on cryptographic hardware and embedded systems, 2018: 238-268.

[30] Rescorla E. The transport layer security (TLS) protocol version 1.3[R]. 2018.

[31] Iyengar J, Thomson M. QUIC: A UDP-based multiplexed and secure transport[R]. RFC 9000, may, 2021.

[32] Kent S, Seo K. Security architecture for the internet protocol[R]. 2005.

[33] Newhouse W, Souppaya M, Barker W, et al. Migration to post-quantum cryptography quantum readi[J]. NIST SPECIAL PUBLICATION, 1800: 38C.

[34] National Institute of Standards and Technology S. FIPS 204, module-lattice-based digital signature standard[J]. 2024.

[35] PUB F. Digital signature standard (DSS)[J]. Fips pub, 2000: 186-192.

[36] Miller V S. Use of elliptic curves in cryptography[C]//Conference on the theory and application of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985: 417-426.

[37] Paillier P. Public-key cryptosystems based on composite degree residuosity classes[C]//International conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999: 223-238.

[38] Kaliski B. RFC1319: The MD2 Message-Digest Algorithm[J]. 1992.

抗量子证书生成工具
测试大纲

编　　写 _____

校　　对 _____

批　　准 _____

文件状态表

文档名称：抗量子证书生成工具测试大纲			
文档版本	修订日期	责任人	修订说明
V1.0	2026.6.9	刘凯博	写测试用例

目　录

1　测试任务 1
1.1　任务概述 1
1.2　研究内容 1
1.3　考核指标 1
1.4　验收测试准则 2

2　被测对象说明 2
2.1　功能概述 2

2.2　测试范围 3

3　依据和引用文件 4
3.1　测试依据 4

4　测试技术及方法 4
4.1　功能测试方法 4
4.2　性能测试方法 4
4.3　兼容性测试方法 5

5　测试环境 5
5.1　硬件环境 5
5.2　软件环境 5

6　考核指标 5

7　问题处理 6

8　测试进度及人员安排 6

9　附录 A 软件问题报告单 7

10　附录 B 验收测试用例集 8
1　抗量子证书生成工具功能测试 9
1.1　混合证书生成功能测试验证 9
1.2　纯抗量子证书生成测试验证 10
1.3　密钥生成与算法支持测试验证 12
1.4　证书字段与扩展验证测试 14
1.5　传统证书链验签测试 16
1.6　外部公钥导入签发测试 18
1.7　文件输出测试 19
2　抗量子证书生成工具性能测试 20
3　抗量子证书生成工具兼容性测试 22
3.1　多算法组合覆盖测试 22
3.2　跨平台编译与运行测试 23
4　测试结果总结 25

1 测试任务

1.1 任务概述

中国科学院信息工程研究所承担《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三"银行业典型信息系统抗量子密码安全协议迁移技术"的研究工作，研制了抗量子证书生成工具。测试组负责对该课题成果进行验收测试，验证其是否满足考核指标要求。

1.2 研究内容

本课题主要开展抗量子证书生成关键技术研究与工具研发工作。针对传统证书体系主要依赖 RSA、SM2 等经典公钥算法、未来可能面临量子计算攻击风险的问题，设计并实现支持传统密码算法、国密算法与抗量子密码算法结合的证书生成工具。工具支持 RSA 证书链、SM2 证书链与抗量子算法扩展结合，也支持基于 ML-DSA/Dilithium、SLH-DSA/SPHINCS、Kyber 等算法构造纯抗量子证书链，为后续抗量子协议测试和证书体系迁移提供基础证书材料。

在证书格式设计方面，本课题基于 X.509 v3 证书结构进行扩展，在保留证书主体、颁发者、有效期、密钥用途、基本约束等标准字段的基础上，增加用于存放抗量子公钥和抗量子签名数据的自定义扩展字段。工具可根据证书用途配置 Key Usage 字段，生成 CA 证书和终端实体证书，并将抗量子算法相关数据写入证书扩展区域，便于后续解析和验签。

通过上述工作，形成抗量子证书生成与验证工具，支持根证书、中间 CA 证书和用户证书的多级证书链生成，支持证书及私钥文件输出，支持传统证书链验签和抗量子签名验签。该工具可为 RSA/SM2 与抗量子算法混合证书、纯抗量子证书，以及抗量子 TLS 等协议适配测试提供证书生成和验证支撑。

1.3 考核指标

根据《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三考核指标 1.2“银行信息系统抗量子/混合密码安全协议库”的要求，本

课题应形成抗量子证书生成工具，支持抗量子/混合证书生成，支持 2 类抗量子密码算法，可达到 128、256 比特的量子安全等级；

1.4 验收测试准则

依据《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》，测试组针对《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三“银行业抗量子密码安全协议库”的研究内容中（2）、考核指标进行测试。验收准则如下：

- 软件系统符合《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三“银行业抗量子密码安全协议库”规定的全部功能要求；
- 验收测试过程中发现的问题全部闭环。

2 被测对象说明

本次测试的被测对象为抗量子证书生成工具。该工具面向银行信息系统抗量子密码迁移场景，用于生成和验证抗量子/混合数字证书，为后续 TLS、IPSec、SSH 等抗量子安全协议适配测试提供证书材料支撑。

被测对象支持 RSA、SM2 等传统/国密算法与 ML-DSA/Dilithium、SLH-DSA/SPHINCS、Kyber 等抗量子算法结合使用，可生成 RSA 与抗量子算法混合证书、SM2 与抗量子算法混合证书，以及纯抗量子证书链。工具基于 X.509 v3 证书结构进行扩展，在保留主体、颁发者、有效期、密钥用途、基本约束等字段的基础上，通过自定义扩展字段写入抗量子公钥和抗量子签名数据。

被测对象的主要功能包括根证书、中间 CA 证书和终端实体证书的多级证书链生成，证书及私钥 PEM 文件输出，外部 SM2 或 ML-DSA 公钥导入签发用户证书，以及证书链解析与验签。测试重点验证不同算法组合下证书是否能够正确生成、保存、解析和验证，传统签名与抗量子签名是否有效，证书扩展字段内容是否符合工具设计要求。

2.1 功能概述

主要功能列表如下：

序号	功能	功能子项	功能描述
1	混合证书链生成	RSA 与抗量子算法混合；SM2 与抗量子算法混合	生成根证书、两级中间 CA 证书和终端实体证书；传统证书签名分别采用 RSA 或 SM2，并在扩展中携带抗量子公钥和签名。
2	纯抗量子证书链生成	Kyber 用户证书链；Dilithium 证书链；SLH-DSA 证书链	生成由 Dilithium 或 SLH-DSA 签名的纯抗量子 X.509 v3 证书链，终端实体可使用 Kyber、Dilithium 或 SLH-DSA 公钥。
3	抗量子密钥生成	Dilithium2/3；SLH-DSA-SHA2-128s/128f；Kyber512/768	生成 CA 所需的抗量子签名密钥，以及终端实体所需的抗量子签名密钥或 Kyber KEM 密钥。
4	证书字段与扩展写入	X.509 v3 基础字段；抗量子公钥扩展；抗量子签名扩展	配置主体、颁发者、有效期、密钥用途和基本约束等字段；在混合证书中写入抗量子公钥及对待签名证书数据生成的抗量子签名。
5	证书链验签	RSA 证书链验签；SM2 证书链验签；纯抗量子证书链验签	验证 RSA 和 SM2 证书链的传统签名关系，并验证 Dilithium 或 SLH-DSA 签名的纯抗量子证书链。
6	证书输出与解析	PEM 证书文件；证书基本信息	将生成的证书保存为 PEM 文件，并解析或输出证书的序列号、签名算法、颁发者、主题、有效期和公钥信息。

2.2 测试范围

本次测试范围覆盖抗量子证书生成工具的核心功能和支撑能力，重点验证工具在混合证书生成、纯抗量子证书生成、密钥生成、证书字段写入、证书校验、外部公钥导入签发和文件输出等方面的实现情况。测试对象包括 RSA 与抗量子算法混合证书、SM2 与抗量子算法混合证书、Kyber 用户证书链、Dilithium 证书链、SPHINCS/SLH-DSA 证书链等生成结果。

功能测试范围包括：验证 RSA、SM2、Dilithium、SPHINCS、Kyber 等算法密钥能否正常生成并用于证书签发；验证根证书、中间 CA 证书和终端实体证书能否按预期生成；验证证书中的主题、颁发者、有效期、Key Usage、Basic Constraints 等基础字段是否正确；验证抗量子公钥和抗量子签名扩展字段是否能够写入、提取并用于验签；验证传统证书链、抗量子签名和纯抗量子证书链的校验结果是否正确。

支撑能力测试范围包括：验证工具能否解析生成证书并输出主要证书信息；验证多种算法组合下的证书生成、输出和验证流程是否完整；统计密钥生成、证

书生成和证书链验证耗时；验证工具在 Linux/Windows 目标环境下的编译运行能力。

3 依据和引用文件

3.1 测试依据

本验收测试大纲所依据和引用的文件见表 3-1。

表 3-1 依据和引用文件

分类	标识	文档名称	版本	发布日期	颁布/来源单位
技术标准	X.509	X.509 — The Directory: Authentication Framework	—	2019.10	CCITT (International Telegraph and Telephone Consultative Committee)
技术标准	GB/T 20518-2018	《信息安全技术 公钥基础设施 数字证书格式》	—	2018.06	国家市场监督管理总局

4 测试技术及方法

4.1 功能测试方法

采用源码配置检查、程序运行输出检查、生成文件检查和证书解析验签等方式，验证抗量子证书生成工具的功能正确性。测试过程中运行各证书生成模块，检查是否能够生成 RSA/SM2 混合证书链、纯抗量子证书链以及外部公钥签发证书；通过程序输出和生成文件确认根证书、中间 CA 证书、终端实体证书及对应私钥是否生成成功；通过证书解析结果检查主题、颁发者、有效期、Key Usage、Basic Constraints、抗量子公钥扩展和抗量子签名扩展等字段是否符合预期；通过传统证书链验签和抗量子签名验签确认生成证书的有效性。

4.2 性能测试方法

采用重复执行密钥生成、证书生成和证书链验证流程的方式，采集不同算法组合下的运行耗时。测试记录包括测试次数、成功次数、失败次数、平均耗时、

最大耗时、最小耗时等数据。测试对象覆盖 RSA、SM2、Dilithium、SPHINCS、Kyber 等算法组合，重点统计密钥生成耗时、证书生成耗时、证书文件输出耗时和证书链验签耗时，用于评估工具在不同抗量子算法组合下的运行开销。

4.3 兼容性测试方法

采用跨环境编译运行和证书格式解析检查的方式，验证工具的工程适配能力和生成结果的可用性。在 Linux、Windows 等目标环境下编译并运行证书生成工具，检查各模块是否能够完成证书生成、文件输出和验签流程。对于 RSA/SM2 混合证书，使用工具自带解析逻辑或通用证书解析工具检查 PEM 文件是否可读取、基础 X.509 字段是否可识别；对于纯抗量子证书，使用工具内置解析与验签逻辑检查证书结构和签发关系是否正确。兼容性测试结果单独记录，用于评估工具在不同运行环境和证书解析场景下的适配情况。

5 测试环境

5.1 硬件环境

名称	型号	备注
笔记本电脑	Dell Precision 3530	处理器：Intel Core i7-8750H*8 内存：16GB 操作系统：Ubuntu 24.04.4 LTS，64 位

5.2 软件环境

名称	软件版本	备注
OpenSSL	1.1.1 或以上版本	RSA/X.509 证书 PEM 格式解析与基础字段检查
Go	go1.22.12 linux/amd64	抗量子证书生成工具编译与运行

6 考核指标

依据验收测试要求进行了全面的需求分析，分解的测试项能够覆盖课题研究相关任务书要求，其追溯关系见下表。

考核指标	完成时指标值/状态	测试设计

指标 1.2 银行信息系统抗量子/混合密码安全协议库	抗量子证书生成工具，支持抗量子/混合证书生成，支持 2 类抗量子密码算法，可达到 128、256 比特的量子安全等级。	测试抗量子证书生成工具对 RSA/SM2 混合证书、纯抗量子证书链、外部公钥导入签发、证书字段与扩展写入、证书 PEM 文件输出、传统证书链验签、抗量子签名验签和纯抗量子证书链验签等功能的支持情况。算法覆盖 Dilithium、SPHINCS、Kyber 等，其中 Kyber-512 对应 128 比特量子安全等级，Kyber-768 对应 256 比特量子安全等级。

7 问题处理

对测试过程中发现的问题进行分析汇总，提交研制方进行问题确认，研制方应对问题进行解释，并提出处理意见（同意修改或不修改），对不修改问题说明理由。研制方根据处理意见对软件进行更改，测试方对更改后的软件完成更改确认和回归测试。

8 测试进度及人员安排

测试进度安排见下表。

序号	测试活动名称	开始日期	结束日期	负责人
1	测试需求分析与测试用例设计	2026.4.15	2026.5.15	刘凯博
2	测试大纲撰写	2026.5.16	2026.6.28	刘凯博
3	测试执行	2026.7.10	2026.7.20	刘凯博
4	回归测试	2026.7.20	2026.8.1	张航
5	测试总结	2026.8.1	2026.8.5	张航

9　附录 A 软件问题报告单

测试问题报告单

测试日期	测试人员	测试问题数量
测试用例标识	测试用例名称	测试问题描述

10 附录 B 验收测试用例集

测试用例集

1 抗量子证书生成工具功能测试

1.1 混合证书生成功能测试验证

测试用例名称	RSA 与抗量子算法混合证书链生成测试	标识	ZS-1-1
测试用例描述	验证抗量子证书生成工具是否能够生成 RSA 与抗量子算法结合的混合证书链。运行 RSA 混合证书生成模块后，检查根证书、中间 CA 证书和终端实体证书是否生成成功，证书中是否写入抗量子公钥和抗量子签名扩展字段，并验证标准 X.509 证书链关系是否正确。		
前提和约束	已安装 Go 运行环境和证书生成工具；已准备 RSA 与抗量子算法混合证书生成模块；测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行 RSA 与抗量子算法混合证书生成程序。	程序运行成功，无异常退出。	
2.	检查输出目录中的证书文件。	成功生成根证书、中间 CA 证书和终端实体证书，证书文件存在且非空。	
3.	解析并验证生成的证书链。	证书基础字段正确，包含抗量子公钥和抗量子签名扩展，标准 X.509 证书链验证通过。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	SM2 与抗量子算法混合证书链生成测试	标识	ZS-1-2
测试用例描述	验证抗量子证书生成工具是否能够生成 SM2 与抗量子算法结合的混合证书链。运行 SM2 混合证书生成模块后，检查根证书、中间 CA 证书、终端实体证书及对应私钥是否生成成功，并验证 SM2 签名关系和抗量子签名关系是否均能通过校验。		
前提和约束	已安装 Go 运行环境和证书生成工具；已准备 SM2 与抗量子算法混合证书生成模块；测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行 SM2 与抗量子算法混合证书生成程序。	程序运行成功，无异常退出。	
2.	检查输出目录中的证书和私钥文件。	成功生成根证书、中间 CA 证书、终端实体证书，以及对应 SM2 私钥和抗量子私钥文件。	
3.	解析并验证生成的证书链。	证书基础字段正确，包含抗量子扩展字段，SM2 证书链验签和抗量子签名验签均通过。	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.2　纯抗量子证书生成测试验证

测试用例名称	Kyber 用户证书链生成测试	**标识**	ZS-1-3
测试用例描述	验证抗量子证书生成工具能够生成 Kyber 用户证书链。该证书链中根证书和中间 CA 证书使用 Dilithium/ML-DSA 类抗量子签名算法完成签发，终端实体证书使用 Kyber 公钥作为用户公钥，用于验证工具对 KEM 类型用户证书的生成、保存、解析和证书链验签能力。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行纯抗量子证书链生成程序。	程序运行成功，无异常退出。	
2.	检查 Kyber 用户证书链输出文件。	成功生成 Kyber 链根证书、中间证书、用户证书及对应私钥文件，文件存在且非空。	
3.	解析并验证 Kyber 用户证书链。	用户证书公钥算法为 Kyber，根证书自签名、中间证书和用户证书签发关系验证通过。	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	Dilithium 证书链生成测试	标识	ZS-1-4
测试用例描述	验证抗量子证书生成工具能够生成 Dilithium/ML-DSA 证书链。该证书链中根证书、中间 CA 证书和终端实体证书均使用 Dilithium/ML-DSA 类抗量子签名算法，测试工具对抗量子签名型证书链的生成、字段写入、文件保存、证书解析和链路验签能力。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行纯抗量子证书链生成程序。	程序运行成功，无异常退出。	
2.	检查 Dilithium 证书链输出文件。	成功生成 Dilithium 根证书、中间证书、用户证书及对应私钥文件，文件存在且非空。	
3.	解析并验证 Dilithium 证书链。	证书签名算法和公钥算法为 Dilithium/ML-DSA 类算法，证书链签发关系验证通过。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	SPHINCS/SLH-DSA 证书链生成测试	标识	ZS-1-5
测试用例描述	验证抗量子证书生成工具能够生成 SPHINCS/SLH-DSA 证书链。该证书链中根证书、中间 CA 证书和终端实体证书均使用 SPHINCS/SLH-DSA 类抗量子签名算法，测试工具对哈希签名类抗量子证书链的生成、文件保存、证书解析和链路验签能力。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行纯抗量子证书链生成程序。	程序运行成功，无异常退出。	
2.	检查 SPHINCS/SLH-DSA 证书链输出文件。	成功生成 SPHINCS/SLH-DSA 根证书、中间证书、用户证书及对应私钥文件，文件存在且非空。	

3.	解析并验证 SPHINCS/SLH-DSA 证书链。	证书签名算法和公钥算法为 SPHINCS/SLH-DSA 类算法，证书链签发关系验证通过。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.3 密钥生成与算法支持测试验证

测试用例名称	RSA、SM2 密钥生成测试	**标识**	ZS-1-6
测试用例描述	验证抗量子证书生成工具能够生成 RSA 和 SM2 密钥，并将其用于对应证书链生成流程。其中 RSA 密钥用于 RSA 混合 X.509 证书链签发，SM2 密钥用于 SM2 混合证书链签发，并可按工具输出规则保存为私钥文件。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行 RSA 混合证书生成程序和 SM2 混合证书生成程序。	程序运行成功，无密钥生成相关异常。	
2.	检查证书生成结果及 SM2 私钥输出文件。	RSA 密钥能够用于证书签发；SM2 私钥文件生成成功，文件存在且非空。	
3.	验证生成证书的签名关系。	RSA 证书链和 SM2 证书链签名验证通过，说明生成密钥可用于证书签发和验签流程。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	Dilithium 签名密钥生成测试	**标识**	ZS-1-7
测试用例描述	验证抗量子证书生成工具能够生成 Dilithium/ML-DSA 签名密钥，并用于抗量子证书签名和验签流程。测试覆盖 Dilithium2、Dilithium3 等签名密钥生成能力，以及其在混合证书扩展签名和纯抗量子证书链签发中的使用		

	情况。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		
测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	运行包含 Dilithium 算法的证书生成程序。	程序运行成功，无 Dilithium 密钥生成异常。	
2.	检查 Dilithium 私钥文件和相关证书文件。	Dilithium 私钥文件及对应证书文件生成成功，文件存在且非空。	
3.	执行 Dilithium 签名验签或证书链验签。	Dilithium 签名验证通过，相关证书链签发关系验证通过。	
测试阶段	☐自测 ☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	SPHINCS 签名密钥生成测试	**标识**	ZS-1-8
测试用例描述	验证抗量子证书生成工具能够生成 SPHINCS/SLH-DSA 签名密钥，并用于纯抗量子证书链生成和签名验证流程。测试覆盖 SPHINCS128s、SPHINCS128f 等算法配置下的密钥生成、证书签发和验签能力。		
前提和约束	已安装 Go 运行环境和证书生成工具，测试环境具备文件写入权限。		
测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	运行包含 SPHINCS/SLH-DSA 算法的证书生成程序。	程序运行成功，无 SPHINCS 密钥生成异常。	
2.	检查 SPHINCS 私钥文件和相关证书文件。	SPHINCS 私钥文件及对应证书文件生成成功，文件存在且非空。	
3.	执行 SPHINCS 证书链验签。	SPHINCS/SLH-DSA 证书链签发关系验证通过。	
测试阶段	☐自测 ☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	Kyber KEM 密钥生成测试	标识	ZS-1-9
测试用例描述	验证抗量子证书生成工具能够生成 Kyber KEM 密钥，并将 Kyber 公钥写入终端实体证书。测试覆盖 Kyber512、Kyber768 等密钥封装算法的密钥生成能力，以及 Kyber 用户证书生成和私钥文件输出情况。		
前提和约束	已安装 Go 运行环境和证书生成工具；已准备包含 Kyber 算法的证书生成模块；测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行包含 Kyber 算法的证书生成程序。	程序运行成功，无 Kyber 密钥生成异常。	
2.	检查 Kyber 用户证书和 Kyber 私钥文件。	Kyber 用户证书及对应私钥文件生成成功，文件存在且非空。	
3.	解析 Kyber 用户证书并验证证书链。	用户证书中包含 Kyber 公钥信息，证书链签发关系验证通过。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

1.4 证书字段与扩展验证测试

测试用例名称	基础证书字段验证测试	标识	ZS-1-10
测试用例描述	验证抗量子证书生成工具生成的证书是否正确写入基础证书字段。测试内容包括证书版本、序列号、主题、颁发者、有效期、CA 标识、Key Usage、Basic Constraints 等字段，确认根证书、中间 CA 证书和终端实体证书的字段设置符合证书链结构要求。		
前提和约束	已安装 Go 运行环境和证书生成工具；已完成混合证书或纯抗量子证书生成；测试环境具备证书解析能力。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行证书生成程序，生成测试证书链。	程序运行成功，生成根证书、中间 CA 证书和终端实体证书。	
2.	解析生成的证书文件。	能够读取证书版本、序列号、主题、颁发者、有效期	

		等基础字段。	
3.	检查证书字段内容。	根证书和中间证书为 CA 证书，终端证书为非 CA 证书，Key Usage、Basic Constraints 等字段符合预期。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	抗量子公钥扩展字段验证测试	**标识**	ZS-1-11
测试用例描述	验证 RSA/SM2 混合证书中是否正确写入抗量子公钥扩展字段。工具在生成混合证书时应通过自定义扩展字段保存抗量子公钥数据，测试需确认该扩展字段存在、内容非空，并能够被工具提取用于后续抗量子签名验证或证书解析。		
前提和约束	已安装 Go 运行环境和证书生成工具；已完成混合证书或纯抗量子证书生成；测试环境具备证书解析能力。		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	运行混合证书生成程序，生成测试证书链。	程序运行成功，生成包含抗量子算法信息的混合证书。	
2.	解析生成证书的扩展字段。	能够读取到抗量子公钥扩展字段，字段内容非空。	
3.	使用工具提取抗量子公钥数据。	抗量子公钥能够被正确提取，并可用于后续抗量子验签流程。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	抗量子签名扩展字段验证测试	**标识**	ZS-1-12
测试用例描述	验证 RSA/SM2 混合证书中是否正确写入抗量子签名扩展字段。工具在生成混合证书时应对证书 TBS 数据生成抗量子签名，并将签名数据写入自定义扩展字段。测试需确认扩展字段存在、签名数据非空，并能够用于抗量子签名验签。		

前提和约束	已安装 Go 运行环境和证书生成工具；已生成 RSA 或 SM2 与抗量子算法混合证书链。		
测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	运行混合证书生成程序，生成测试证书链。	程序运行成功，生成包含抗量子签名扩展的混合证书。	
2.	解析生成证书的扩展字段。	能够读取到抗量子签名扩展字段，字段内容非空。	
3.	使用扩展字段中的抗量子签名执行验签。	抗量子签名能够通过验证，说明签名扩展字段写入正确。	
测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

1.5 传统证书链验签测试

测试用例名称	传统证书链验签测试	标识	ZS-1-13
测试用例描述	验证抗量子证书生成工具生成的 RSA/SM2 混合证书链是否能够完成传统证书链验签。测试内容包括根证书自签名验证、中间 CA 证书由上级证书签发验证、终端实体证书由中间 CA 证书签发验证，用于确认传统签名体系下证书链关系正确。		
前提和约束	已安装 Go 运行环境和证书生成工具;已生成 RSA 或 SM2 与抗量子算法混合证书链。		
测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	运行 RSA 或 SM2 混合证书生成程序。	程序运行成功，生成根证书、中间 CA 证书和终端实体证书。	
2.	使用工具自带验签逻辑或证书解析工具验证证书链。	根证书自签名、中间证书签发关系、终端证书签发关系能够被正确识别。	
3.	查看证书链验签结果。	传统证书链验签通过,无签名错误或链路断裂。	
测试阶段	□自测 □第三方测试		

通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	抗量子签名验签测试	**标识**	ZS-1-14
测试用例描述	验证 RSA/SM2 混合证书中抗量子签名是否能够正确验签。工具在生成混合证书时将抗量子公钥和抗量子签名写入证书扩展字段，测试通过提取扩展字段中的抗量子公钥和签名，对证书原始 TBS 数据执行抗量子签名验证，确认抗量子签名数据与证书内容匹配。		
前提和约束	已安装 Go 运行环境和证书生成工具；已生成包含抗量子公钥扩展和抗量子签名扩展的混合证书链。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行包含抗量子签名扩展的混合证书生成程序。	程序运行成功，生成包含抗量子公钥和抗量子签名扩展的证书。	
2.	提取证书中的抗量子公钥、抗量子签名和 TBS 数据。	能够正确提取抗量子公钥、抗量子签名和待验签数据。	
3.	使用工具执行抗量子签名验签。	抗量子签名验签通过，程序输出抗量子验签成功。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	纯抗量子证书链验签测试	**标识**	ZS-1-15
测试用例描述	验证纯抗量子证书链是否能够完成链路验签。测试对象包括 Kyber 用户证书链、Dilithium 证书链和 SPHINCS/SLH-DSA 证书链。工具应能够解析纯抗量子证书结构，并验证根证书自签名、中间证书由根证书签发、终端实体证书由中间证书签发的链路关系。		
前提和约束	已安装 Go 运行环境和证书生成工具；已生成 Kyber、Dilithium、SPHINCS/SLH-DSA 等纯抗量子证书链。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行纯抗量子证书链生成程序。	程序运行成功，生成纯抗量子证书链文件。	
2.	使用工具解析纯抗量子证书链。	能够解析根证书、中间证书和	

		终端实体证书的结构及算法信息。	
3.	执行纯抗量子证书链验签。	根证书自签名、中间证书签发关系、终端证书签发关系均验证通过。	
测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.6 外部公钥导入签发测试

测试用例名称	SM2 外部公钥导入签发测试	**标识**	ZS-1-16
测试用例描述	验证抗量子证书生成工具能够从外部公钥文件中读取 SM2 用户公钥，并生成由本地 SM2 根证书签发的用户证书。该测试用于确认工具支持用户私钥不导出的签发场景，即用户私钥由对接方持有，证书生成工具仅导入用户公钥完成证书签发。		
前提和约束	已安装 Go 运行环境和证书生成工具；已准备包含 SM2 公钥参数的外部公钥文件；测试环境具备文件读取和写入权限。		
测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	准备包含 SM2 公钥信息的外部公钥文件。	外部公钥文件格式符合工具读取要求，包含 SM2 公钥 X、Y 参数。	
2.	运行 SM2 外部公钥导入签发程序。	程序能够成功读取外部 SM2 公钥，并生成 SM2 根证书和用户证书。	
3.	验证生成的 SM2 用户证书。	用户证书公钥与导入的 SM2 公钥一致，用户证书由 SM2 根证书签发且验签通过。	
测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	ML-DSA 外部公钥导入签发测试	**标识**	ZS-1-17
测试用例描述	验证抗量子证书生成工具能够从外部公钥文件中读取 ML-DSA 用户公钥，		

	并生成由本地 ML-DSA 根证书签发的用户证书。该测试用于确认工具支持纯抗量子证书场景下的外部公钥导入签发，用户私钥不由证书生成工具导出。
前提和约束	已安装 Go 运行环境和证书生成工具；已准备包含 ML-DSA 公钥的外部公钥文件；测试环境具备文件读取和写入权限。

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	准备包含 ML-DSA 公钥信息的外部公钥文件。	外部公钥文件格式符合工具读取要求，包含 ML-DSA 公钥数据。	
2.	运行 ML-DSA 外部公钥导入签发程序。	程序能够成功读取外部 ML-DSA 公钥，并生成 ML-DSA 根证书和用户证书。	
3.	验证生成的 ML-DSA 用户证书。	用户证书公钥与导入的 ML-DSA 公钥一致，用户证书由 ML-DSA 根证书签发且验签通过。	

测试阶段	□自测　□第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.7　文件输出测试

测试用例名称	证书 PEM 文件输出测试	**标识**	ZS-1-18
测试用例描述	验证抗量子证书生成工具能够将生成的证书保存为 PEM 文件。测试内容包括 RSA/SM2 混合证书、纯抗量子证书链和外部公钥签发证书的输出文件，确认生成文件存在、内容非空、PEM 块类型正确，并可用于后续证书解析或验签。		
前提和约束	已安装 Go 运行环境和证书生成工具；已运行对应证书生成模块；测试环境具备文件写入权限。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	运行证书生成程序。	程序运行成功，生成证书输出文件。	
2.	检查输出目录中的证书 PEM 文件。	证书文件存在且非空，PEM 块类型为 CERTIFICATE。	

3.	使用工具解析生成的证书文件。	证书文件能够被读取和解析，可用于后续证书字段检查或验签。	
测试阶段	□自测　□第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	私钥 PEM 文件输出测试	**标识**	ZS-1-19
测试用例描述	验证抗量子证书生成工具能够将生成的私钥保存为 PEM 文件。测试内容包括 SM2 私钥、Dilithium/ML-DSA 私钥、SPHINCS/SLH-DSA 私钥和 Kyber 私钥等输出结果，确认私钥文件存在、内容非空、PEM 块类型正确，并可被工具后续流程使用。		
前提和约束	已安装 Go 运行环境和证书生成工具；已运行支持私钥输出的证书生成模块；测试环境具备文件写入权限。		
测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	运行支持私钥输出的证书生成程序。	程序运行成功，生成对应私钥文件。	
2.	检查输出目录中的私钥 PEM 文件。	私钥文件存在且非空，PEM 块类型为 PRIVATE KEY。	
3.	使用工具后续流程验证私钥可用性。	私钥能够用于对应证书生成、签名、验签或密钥封装相关流程，无格式错误。	
测试阶段	□自测　□第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

2　抗量子证书生成工具性能测试

测试用例名称	密钥生成性能测试	**标识**	ZS-2-1
测试用例描述	验证抗量子证书生成工具在不同算法下的密钥生成效率。测试对象包括 RSA、SM2、Dilithium/ML-DSA、SPHINCS/SLH-DSA、Kyber 等算法，记录各类密钥生成耗时，用于评估工具在不同算法组合下的运行开销。		
前提和约束	已安装 Go 运行环境和证书生成工具；测试环境硬件和软件配置固定；测试期间无其他高负载任务干扰。		

测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	分别运行包含 RSA、SM2、Dilithium、SPHINCS、Kyber 密钥生成流程的程序。	各算法密钥均能生成成功，无异常退出。	
2.	重复执行密钥生成流程并记录耗时。	记录测试次数、成功次数、失败次数、平均耗时、最大耗时、最小耗时。	
3.	汇总不同算法密钥生成耗时。	密钥生成性能满足测试要求，未出现明显异常耗时或生成失败。	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	证书生成性能测试	**标识**	ZS-2-2
测试用例描述	验证抗量子证书生成工具在不同证书类型下的证书生成效率。测试对象包括 RSA 与抗量子算法混合证书链、SM2 与抗量子算法混合证书链、Kyber 用户证书链、Dilithium 证书链和 SPHINCS/SLH-DSA 证书链，记录生成耗时并分析不同算法组合对证书生成性能的影响。		
前提和约束	已安装 Go 运行环境和证书生成工具；测试环境硬件和软件配置固定；输出目录可写；测试期间无其他高负载任务干扰。		

测试步骤			
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	分别运行不同类型证书链生成程序。	各类型证书链均能生成成功，无异常退出。	
2.	重复执行证书生成流程并记录耗时。	记录测试次数、成功次数、失败次数、平均耗时、最大耗时、最小耗时。	
3.	汇总不同证书链生成耗时。	证书生成性能满足测试要求，输出文件完整，未出现明显异常耗时或生成失败。	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	证书链验证性能测试	标识	ZS-2-3
测试用例描述	验证抗量子证书生成工具在不同证书链类型下的证书链验证效率。测试内容包括传统 RSA/SM2 证书链验签、抗量子签名验签和纯抗量子证书链验签，记录各类证书链验证耗时，用于评估工具在证书校验环节的运行开销。		
前提和约束	已安装 Go 运行环境和证书生成工具；已生成待验证的混合证书链和纯抗量子证书链；测试环境硬件和软件配置固定；测试期间无其他高负载任务干扰。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	准备待验证的 RSA/SM2 混合证书链和纯抗量子证书链。	待验证证书链文件存在且可解析。	
2.	重复执行证书链验签流程并记录耗时。	记录测试次数、成功次数、失败次数、平均耗时、最大耗时、最小耗时。	
3.	汇总不同证书链验证耗时。	各类证书链验证均通过，验证性能满足测试要求，未出现明显异常耗时或验证失败。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

3 抗量子证书生成工具兼容性测试

3.1 多算法组合覆盖测试

测试用例名称	多算法组合覆盖测试	标识	ZS-3-1
测试用例描述	验证抗量子证书生成工具对多种算法组合的覆盖情况。测试内容包括 RSA 与抗量子算法混合证书、SM2 与抗量子算法混合证书、Kyber 用户证书链、Dilithium 证书链、SPHINCS/SLH-DSA 证书链等，确认工具能够覆盖传统算法、国密算法、抗量子签名算法和抗量子 KEM 算法的主要组合场景。		
前提和约束	已安装 Go 运行环境和证书生成工具；测试环境具备文件写入权限；相关证书生成模块可正常运行。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	分别运行 RSA 混合证书、SM2 混合证书和纯抗	各证书生成程序均运行成	

	量子证书生成程序。	功，无异常退出。	
2.	检查不同算法组合下的输出文件。	成功生成 RSA、SM2、Dilithium、SPHINCS、Kyber 等算法组合对应的证书文件。	
3.	验证各类证书链生成和验签结果。	各算法组合下证书链生成成功，证书解析和验签结果符合预期。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

3.2 跨平台编译与运行测试

测试用例名称	Linux 环境编译运行测试	**标识**	ZS-3-2
测试用例描述	验证抗量子证书生成工具在 Linux 环境下能够完成编译、运行、证书生成、文件输出和验签流程。测试覆盖主要证书生成模块，用于确认工具在 Linux 目标环境下具备基本工程适配能力。		
前提和约束	已安装 Go 运行环境；已获取抗量子证书生成工具源码；Linux 环境具备编译、运行和文件写入权限。		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1	在 Linux 环境下编译证书生成工具。	编译成功，无编译错误。	
2	运行主要证书生成程序。	程序运行成功，能够生成证书和私钥文件。	
3	执行证书解析和验签流程。	证书可解析，证书链和签名验证结果符合预期。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	Windows 目标平台交叉编译测试	**标识**	ZS-3-3
测试用例描述	验证抗量子证书生成工具能够在 Linux 环境下生成 Windows 目标平台可执行文件，用于确认工具具备 Windows 目标平台的基础工程适配能力。测试内容包括配置 Windows 目标平台编译参数、执行交叉编译、检查可		

	执行文件是否生成。			
前提和约束	已安装 Go 运行环境；已获取抗量子证书生成工具源码；Linux 环境具备编译权限；源码依赖满足交叉编译要求。			
测试步骤				
序号	**输入及操作**	**期望结果与评估标准**	**实测结果**	
1	在 Linux 环境下配置 Windows 目标平台交叉编译参数。	编译参数配置成功，目标平台设置为 Windows。		
2	执行 Windows 目标平台交叉编译。	编译过程无错误退出。		
3	检查交叉编译输出文件。	成功生成 Windows 可执行文件，文件存在且非空。		
测试阶段	☐自测　☐第三方测试			
通过情况		**测试人员**		
测试时间		**测试地点**		

4 测试结果总结

序号	测试用例	测试结果
1	ZS-1-1	□通过 □不通过
2	ZS-1-2	□通过 □不通过
3	ZS-1-3	□通过 □不通过
4	ZS-1-4	□通过 □不通过
5	ZS-1-5	□通过 □不通过
6	ZS-1-6	□通过 □不通过
7	ZS-1-7	□通过 □不通过
8	ZS-1-8	□通过 □不通过
9	ZS-1-9	□通过 □不通过
10	ZS-1-10	□通过 □不通过
11	ZS-1-11	□通过 □不通过
12	ZS-1-12	□通过 □不通过
13	ZS-1-13	□通过 □不通过
14	ZS-1-14	□通过 □不通过
15	ZS-1-15	□通过 □不通过
16	ZS-1-16	□通过 □不通过
17	ZS-1-17	□通过 □不通过
18	ZS-1-18	□通过 □不通过
19	ZS-1-19	□通过 □不通过
20	ZS-2-1	□通过 □不通过
21	ZS-2-2	□通过 □不通过
22	ZS-2-3	□通过 □不通过
23	ZS-3-1	□通过 □不通过
24	ZS-3-2	□通过 □不通过
25	ZS-3-3	□通过 □不通过
测试结果统计		测试___项，通过___项